

基于末端覆盖的无人机投弹策略研究

摘 要

随着精确制导武器的迅速发展,导弹在末端阶段对地面目标的威胁日益突出。本文围绕如何在导弹飞行全过程中实现持续有效的遮蔽,逐步建立从单机单弹到多机多弹协同的系统化模型,并在各阶段引入针对性的优化算法。

针对问题一,本文提出导弹一烟幕一目标的几何判据,以最小距离条件作为完全遮蔽的判断标准。算法上采用**候选点精简**与**事件驱动扫描**结合**二分法精化**的方式,快速求解遮蔽区间,保证了判据计算的严谨性与高效性。最终得出干扰弹对导弹 M1 的有效遮蔽时长为 **1.406250 s**。

针对问题二,本文将无人机航向、速度及投放时机作为优化变量,建立最大化遮蔽时长的时空匹配模型。算法上采用**三层启发式搜索**结合**黄金分割精化**:外层对航向与速度做粗网格搜索,中层对投放时刻进行枚举,内层利用黄金分割法优化引信延迟,并通过局部爬山精化边界,从而获得近似全局最优解:总遮蔽时长为 **4.5853 s**,具体数据详见表二。

针对问题三,本文研究单机多弹的连续覆盖。针对区间组合复杂性,提出**候选库生成一约束贪心选择一交替一维精化**的混合算法。通过预生成高质量候选解、在最小间隔约束下贪心选择三枚弹,再在局部进行**一维黄金分割精化**,有效实现连续覆盖时长最大化。最终得出有效遮蔽总时长: **6.399 s**,具体数据详见表三。

针对问题四,本文扩展到三机协同,提出“**末端覆盖协同防护模型**”。该模型先由 FY3 在末端阶段实现完全覆盖,再由 FY1 与 FY2 在前端错峰补充。算法上采用**两步法**结合**大邻域搜索 LNS**:第一步固定 FY3 的末端覆盖解,第二步在前端生成候选并通过贪心与 LNS 搜索实现前后衔接,从而显著延长整体防护时间,具体结果计算得出总遮蔽时长为 **26.9352s**,具体数据详见表四。

针对问题五,本文将场景推广至多机多导弹,构建“**航线选择一投放时序**”**双层优化**框架。上层通过**几何指标**生成无人机航向一速度候选,并赋予几何一致性奖励;下层将投放事件离散化,采用带奖励的**最大覆盖贪心**结合**局部搜索**近似求解,同时可用整数线性规划在小规模下获得精确解。该算法兼顾计算可行性与求解精度,实现多机多弹的全局协同防护。经计算 M1、M2、M3 遮蔽总时长分别为 **25.2951s**、**23.0492s**、**15.66769s**,具体数据见表六。

关键字: 末端覆盖 遮蔽判据模型 黄金分割精化 启发式搜索

一、问题重述

1.1 问题背景

现代战争中，精确制导武器的发展使得防御方目标面临极大威胁。空地导弹凭借高速飞行和精确制导能力，可以在极短时间内打击目标，传统的防护手段往往难以及时应对。为提高防御系统的生存能力，烟幕干扰技术作为一种低成本、高效益的对抗手段受到广泛关注。其基本原理是通过投放烟幕干扰弹，在目标与来袭导弹之间形成遮蔽云团，从而干扰导弹的探测与制导系统，降低目标被发现和命中的概率。

随着无人机技术的快速发展，利用无人机实施烟幕干扰成为一种可行且灵活的策略。无人机具有机动性强、任务执行灵活、可多机协同的优势，可以在导弹来袭时迅速进入目标空域并精确投放烟幕干扰弹。然而，如何合理设计无人机的飞行路径、飞行速度以及烟幕干扰弹的投放与起爆时机，是实现有效遮蔽的关键问题。

具体而言，烟幕干扰弹在脱离无人机后受重力作用运动，并在起爆瞬间形成球状云团。该云团具有下沉与弥散特性，其遮蔽效果存在时效性与空间范围的限制。因此，如何通过精确控制投放点和起爆时刻，使得烟幕在导弹飞行路径与目标之间形成有效遮蔽是问题的核心。若投放策略不合理，烟幕可能无法覆盖导弹视线，或遮蔽时间不足，导致防护效果显著下降。

1.2 问题重述

在本题设定中，导弹以 300m/s 的高速逼近，目标体积有限，而无人机数量、速度和投放频率均受到约束。因此，需要构建合理的数学模型，综合考虑导弹轨迹、无人机运动学特性以及烟幕的空间演化规律，从而设计最优的投放策略，尽可能延长真目标的有效遮蔽时间。

根据题设，我们把假目标作为原点，把水平面设为 xy 平面，真目标作为圆柱体其下底面圆心坐标为 $(0, 200, 0)$ 。当警戒雷达发现导弹时，3 枚导弹 M1、M2、M3 坐标分别为 $(20000, 0, 2000)$ 、 $(19000, 600, 2100)$ 、 $(18000, -600, 1900)$ ；5 架无人机坐标别为 FY1 $(17800, 0, 1800)$ 、FY2 $(12000, 1400, 1400)$ 、FY3 $(6000, -3000, 700)$ 、FY4 $(11000, 2000, 1800)$ 、FY5 $(13000, -2000, 1300)$ 。具体三维图例如下：

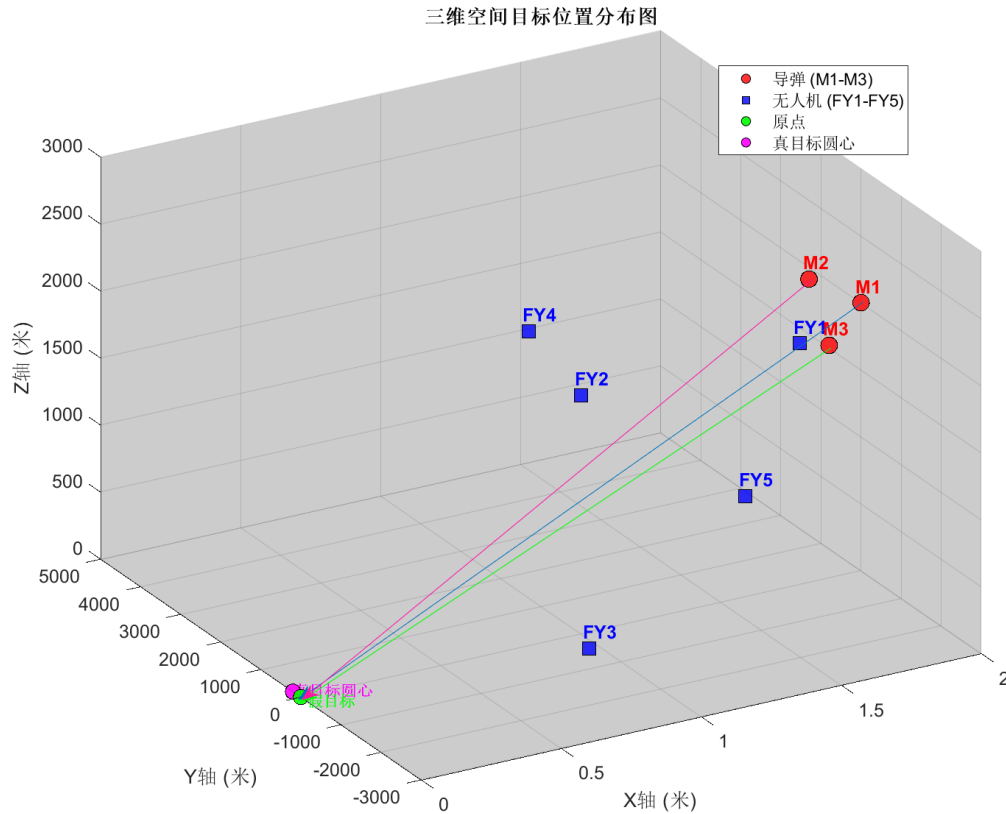


图 1：导弹与无人机的三维空间位置分布图

问题一：考虑无人机 FY1 从位置 (17800, 0, 1800) 出发，以 120m/s 的速度朝假目标方向飞行，其在接收任务 1.5s 后投放一枚烟幕干扰弹，该干扰弹仅受重力作用下落，并在投放 3.6s 后起爆形成球状烟幕云团。云团在起爆瞬间生成，随后以 3m/s 的速度匀速下沉，且在起爆后 20s 内，其中心 10m 范围内可对目标提供有效遮蔽。与此同时，导弹 M1 从 (20000, 0, 2000) 出发，以 300m/s 的速度沿直线飞向假目标。问题要求结合导弹与云团的时空轨迹，计算烟幕干扰弹对导弹 M1 的有效遮蔽时长。

问题二：与问题一不同，本题需综合优化无人机的飞行方向、飞行速度、干扰弹投放点以及起爆点等关键参数，使得尽可能延长遮蔽持续时间，从而最大化提升对真目标的防护效果。

问题三：本问题中干扰弹由 1 枚变为 3 枚，需要基于第二问的讨论给出最优的投放策略，进一步延长完全遮蔽的持续时间。

问题四：在前述单机对抗的基础上，现又引入两架无人机协同防护真目标。三架无人机可分别规划飞行航向与速度，并各自投放 1 枚烟幕干扰弹。需综合优化三架无人机的协同路径与投放方案，使生成的多个烟幕云团在时空上形成合理覆盖，以延长导弹 M1 飞向假目标过程中的整体遮蔽持续时间，从而进一步提升防护效果。

问题五：在前四问的基础上，进一步考虑导弹数量增加、来袭方向不同等更复杂的攻击场景。要求在 5 架无人机且各至多投掷 3 枚干扰弹的约束下，设计能够应对 3 枚导弹威胁的防护方案。

二、问题分析

2.1 问题一的分析

我们需要分析无人机 FY1 投放一枚烟幕干扰弹对导弹 M1 的遮蔽效果。整个场景可以抽象为三个核心要素：直线等速飞向假目标的导弹、位于目标区域的真目标圆柱体，以及由无人机投放并延时起爆形成的球状烟幕云团。无人机按照既定航向和速度飞行，在预设的时间投放干扰弹，干扰弹下落一段时间后在指定时刻起爆，形成的烟幕云团随后以恒定速度下沉，并在有限的时间内具备有效遮蔽能力。问题最为关键的一点是如何判断烟幕云团有效遮蔽：当导弹飞向真目标的视线与烟幕云团发生相交，使得真目标整体处于遮挡状态时，记为有效遮蔽。由于真目标的尺度远小于导弹与云团之间的距离，可近似认为其处于导弹视线束的一个有限区域内，只有当云团中心轨迹在空间上足够接近这一通道并且时间上与云团的有效期重合时，才会产生遮蔽效果。基于这一逻辑，可以在起爆时刻之后沿时间轴考察导弹与云团的相对位置关系，从而确定遮蔽发生的起始时刻与结束时刻，进而得到有效遮蔽的持续时间。这同时也揭示了影响遮蔽效果的关键因素有烟幕起爆位置、云团下沉轨迹与导弹视线束的对齐程度，以及云团有效期与导弹飞行的时间匹配，为第二问作出铺垫。

2.2 问题二的分析

问题二研究的核心目标是确定无人机 FY1 的最优投放策略，使烟幕干扰弹能够产生有效遮蔽。与问题一中飞行速度与投放时机确定的情形不同，此处无人机的飞行方向、飞行速度、投放点以及起爆点均为可调节参数，因而问题的复杂性显著提高。建模首先要明确约束条件，其中包括无人机的飞行速度范围、投放间隔要求以及烟幕干扰弹的下落特性。导弹的速度与飞行方向则决定了其到达目标的时间与位置窗口，这为遮蔽的时空需求提供了边界条件。其次，需要分析烟幕干扰弹在脱离无人机后的运动过程以及起爆后烟幕云团的扩散与下沉特性，从而确定其对目标的有效覆盖范围及持续时间。由此可见，本问题的关键在于实现时空匹配：无人机需要在适当的时刻和位置投放干扰弹，使得烟幕云团能够在导弹到达真目标附近的关键区间内遮蔽目标。最终，该问题可抽象为一个以最大化遮蔽时间为目标的多参数优化问题，涉及飞行路径规划、投放时机设计以及遮蔽效果的综合评估。而对于真目标被完全遮蔽的判断策略已由问题一给出。

2.3 问题三的分析

问题三要求无人机投放三枚烟幕干扰弹，对来袭导弹形成持续遮蔽。关键在于合理安排投放与起爆的时序和位置，使目标在导弹接近的关键阶段尽可能长时

间处于遮蔽状态。导弹轨迹与到达时间是确定的，因此需要根据其接近目标的时间窗口，依次布设烟幕。无人机的飞行参数一旦确定便保持不变，这使投放策略必须在初始阶段统一规划。烟幕形成后持续时间有限，三次起爆应在时间上衔接、空间上覆盖导弹路径，同时满足投放间隔的约束。因此，该问题可视为一个时间与空间协调的优化过程。核心思路是：先确定目标暴露的关键时段，再规划无人机飞行与投放点，最后通过优化三次起爆顺序与时机，实现遮蔽区间的最大化覆盖，从而延长整体干扰效果。

2.4 问题四的分析

问题四要求三架无人机协同投放烟幕干扰弹以干扰来袭导弹 M1，其关键在于合理规划飞行路径与投放时机，使烟幕云团在真目标暴露的关键时段形成有效遮蔽。在模型建立中，我们沿用问题二、三的遮蔽判据模型；并且经初步计算我们发现在导弹抵达目标之前无人机 FY3 可以将干扰弹投掷到真目标上顶面圆心处，故可以做到烟幕将真目标完全包裹，从而在末端阶段实现对真目标的完全覆盖，这大大简化了模型建立。我们只要在问题二的基础上讨论 FY1 及 FY2 即可。我们采用时间区间反推与空间几何分析相结合的方法，明确遮蔽所需的时空条件，并通过协同优化设计三架无人机的投放次序与起爆时刻，使烟幕在时间上实现错峰衔接，在空间上形成互补覆盖。最终通过数值模拟验证方案的有效性和遮蔽持续时间。整体而言，该问题实质上是一个多约束条件下的协同调度优化，需要在导弹飞行特性、无人机运动学限制和烟幕物理特性之间寻求平衡，以实现真目标的持续防护。

2.5 问题五的分析

问题五要求利用五架无人机对三枚来袭导弹M1、M2、M3进行协同干扰。相比问题四，难点在于导弹数量增加，需在有限投放次数下实现多目标的遮蔽。在建模中，我们继续采用烟幕遮蔽判据模型来判定防护效果，并结合导弹飞行时间预测推算关键时段。由于无人机数量增加，我们建立了无人机任务分配与路径规划决策模型，在速度和航向约束下合理分配投放任务。在策略设计上，我们继续采用“末端覆盖的协同防护模型，”同时继续利用时间区间反推与空间几何分析方法，并结合协同优化调度，使多架无人机在时间上错峰衔接、空间上互补覆盖。整体来看，该问题本质上是一个多目标、多约束的组合优化问题，需在导弹特性、无人机运动限制和烟幕特性之间平衡，以实现真目标的有效防护。

三、模型假设

1. 坐标系固定，假目标为原点，设竖直向上方向为 z 轴。
2. 无人机在投放瞬时将干扰弹以无人机速度释放，相对速度忽略不计。
3. 起爆瞬间即形成理想球形烟幕云团，球心仅做竖直匀速下沉，无横向漂移。
4. 忽略大气扩散、风造成的云团扩散或形状变化。

5. 云团与地面接触瞬间由球体近似变为边长为 $10m$ 的正方体。

四、符号说明

表 1 符号说明

符号	说明
F	假目标坐标
T_0	真目标底面圆心坐标
R_T, H_T	真目标底面半径、高度
$\mathcal{T}$	真目标的目标集合
M_0	导弹初始位置
v_M	导弹速度
$ \cdot $	范数
t_{rel}	投放时刻
Δ	引信延迟
t_e	起爆时刻
t_{imp}	终止时刻
C_0	起爆瞬时球心初始位置
R_s	云团半径
v_s	球心下沉速度
T_s	烟幕有效期
g	重力常数

五、模型的建立与求解

5.1 问题一的求解

5.1.1 模型的建立

我们的目标是在任意时刻 t ，判断来袭导弹 M_i 是否对整个真目标圆柱体 $\mathcal{T}$ 实现完全遮蔽，即对任意真目标点 $P \in \mathcal{T}$ ，存在导弹到该点的视线段在到达 P 之前与烟幕球体相交。

Step1 运动学方程建立

我们给出 t 时刻导弹的位置函数：

$$M(t) = M_0 + v_M \widehat{u}_M t, \quad \widehat{u}_M = \frac{F - M_0}{|F - M_0|}$$

竖直方向我们考虑自由落体运动，则无人机投放后干扰弹在 $t \in [t_{rel}, t_e]$ 位置函数：

$$G(t) = U(t_{rel}) + v_{G0}(t - t_{rel}) + \frac{1}{2}(0, 0, -g)(t - t_{rel})^2$$

其中 v_{G0} 为无人机速度向量。

烟幕球心在 $t \in [t_e, t_e + T_s]$ 时的位置函数：

$$C(t) = C_0 + (0, 0, -v_s)(t - t_e)$$

此外，我们给出无人机、导弹、干扰弹、云团的运动示意图如下：

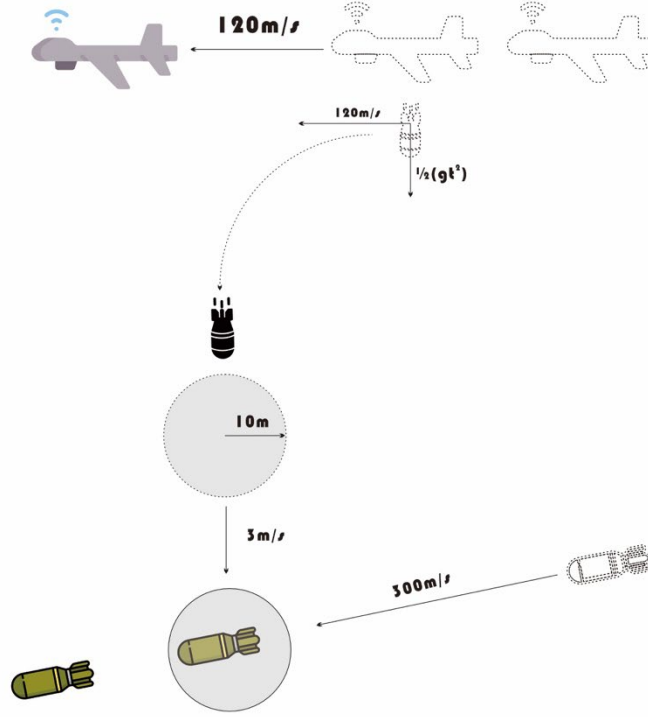


图 2: 无人机、导弹、干扰弹、云团的运动示意图

Step2 完全遮蔽的等价命题

在任意时刻 $t \in [t_e, t_e + T_s]$, 我们有如下等价命题来判断真目标被完全遮蔽:

(A) 对于任意点 $P \in \mathcal{T}$, 存在 $s \in [0, 1]$, $s.t.$

$$|M(t) + s(P - M(t)) - C(t)| \leq R_s$$

那么真目标就被完全遮蔽。

(B) 我们定义

$$d_{min}(P; t) := \min_{s \in [0, 1]} ||M(t) + s(P - M(t)) - C(t)||$$

则真目标被完全遮蔽当且仅当

$$\max_{P \in \mathcal{T}} d_{min}(P; t) \leq R_s$$

命题 B 是对命题 A 的优化, 由此判定“完全遮蔽”问题等价于计算连续目标集合上的上确界并与 R_s 比较。

Step3 单点到球心的“段最短距离”解析解

对任意固定 $P \in \mathcal{T}$ 与时刻 t , 我们记

$$a(t) = M(t) - C(t), \quad d(P, t) = P - M(t)$$

由 Step2 的讨论我们即要计算

$$d_{min}^2(P; t) = \min_{s \in [0, 1]} ||a(t) + s d(P, t)||^2$$

展开二次函数

$$\phi(s) = |a + sd|^2 = As^2 + 2Bs + C, \text{ 其中 } A = |d|^2, B = a \cdot d, C = |a|^2$$

为便于书写, 此处省略了对 t 与 P 的显式依赖。我们直接由二次函数极值公式与端点比较可以得出, 对实数 s 的无约束极值位于

$$s^* = -\frac{B}{A} = -\frac{a \cdot d}{|d|^2}$$

因此对约束 $s \in [0,1]$,

$$d_{min}^2(P; t) = \begin{cases} |a|^2, & s^* \leq 0 \text{ (取得于 } s = 0) \\ |a|^2 - \frac{(a \cdot d)^2}{|d|^2}, & 0 < s^* < 1 \text{ (取得于 } s = s^*) \\ |a + d|^2 = |P - C|^2, & s^* \geq 1 \text{ (取得于 } s = 1) \end{cases}$$

即当投影点落在线段内部($s^* \in (0,1)$)时, 最近点为线段内垂足; 若投影点在端点外, 则最近点为端点($s = 0$ 或 $s = 1$)。

Step4 问题再优化

由前述讨论, 在时刻 t 真目标被完全遮蔽等价于

$$\max_{P \in \mathcal{T}} d_{min}^2(P; t) \leq R_s^2$$

为便于计算我们把真目标上的点集参数化:

$$P(\theta, h) = (R_T \cos \theta, y_T + R_T \sin \theta, h), \quad \theta \in [0, 2\pi), h \in [0, H_T]$$

于是问题变为求解二维连续优化问题

$$\max_{\theta \in [0, 2\pi), h \in [0, H_T]} \Psi(\theta, h; t) \leq R_s^2, \text{ 其中 } \Psi(\theta, h; t) = d_{min}^2(P(\theta, h); t)$$

当然, 由于函数 Ψ 在紧致集合 $[0, 2\pi] \times [0, H_T]$ 上连续, 最大值存在。

Step5 求最大值的结构性分析

为构造可行而有限的判定流程, 我们需要刻画 Ψ 的临界点与边界最大点的性质, 从而把连续优化化归为有限候选点的比较或一维根求解。下面我们将给出两个相关引理。

Lemma1. 对光滑区间 $0 < s^* < 1$, 函数 $\Psi(\theta, h)$ 可写为

$$\Psi(\theta, h) = |a|^2 - \frac{(a \cdot d)^2}{|d|^2}$$

设在该区间上取极值且为内点, 即对 θ, h 的偏导数为零, 对变量 d 的无约束梯度满足

$$\nabla_d \left(\frac{(a \cdot d)^2}{|d|^2} \right) = 0$$

对函数

$$q(d) = \frac{(a \cdot d)^2}{|d|^2}$$

有梯度

$$\nabla_d q = \frac{2(a \cdot d)}{|d|^4} (|d|^2 a - (a \cdot d)d)$$

令其为零向量得两类可能：

- (1) $a \cdot d = 0$ ，表示 d 与 a 正交，对应 $s^* = 0$ 的边界情形，此时 $\Psi = |a|^2$ 。
- (2) $|d|^2 a = (a \cdot d)d$ ，即 d 与 a 平行，这意味着极值候选点必位于通过 M 且方向为 a 的直线与圆柱体 $\mathcal{T}$ 的交点上，其中交点个数有限最多为两个。

因此在内部区间的内点临界点要么退化为边界，即 $s^* = 0$ ，要么为“直线-圆柱交点”类型，且交点有限。

Lemma2. 对边界情形，当 $s^* \geq 1$ 时， $\Psi(\theta, h) = |P - C|^2$ ，该表达在给定 h 或 θ 下关于另一变量是连续可微的，故其最大值必发生在相应边界，即在 h 的端点 0 或 H_T 或在 θ 的临界点由方程 $\partial \Psi / \partial \theta = 0$ 给出。当 $s^* \leq 0$ 时， $\Psi(\theta, h) = |a|^2$ 与 (θ, h) 无关，则 $\max \Psi = |a|^2$ 。

Step6 判定完全遮蔽的有限候选点法

在给定时刻 t ，判断完全遮蔽只需比较下列有限或可数个候选点处的值并取最大值：若 $|a(t)| \leq R_s$ ，此时导弹位于云团内部，则立即判定为完全遮蔽；否则，我们需按如下方法构造候选集 $\mathcal{P}_{cand}$ ：

(A) 圆柱顶面与底面边界处的极值点：即参数 (θ, h) 中 $h \in \{0, H_T\}$ 时对 θ 求解 $\partial_\theta \Psi = 0$ 或直接取 θ 使得点到 C 在 xy 平面上距离最大。

(B) 圆柱侧面上与直线 $L: M + \lambda a$ 的交点（若存在）：求解

$$P = M + \lambda a, \text{ 且 } |P_{xy} - T_{0,xy}| = R_T, 0 \leq P_z \leq H_T$$

求解该二次方程得到至多两个 λ ，相应交点构成候选点，这对应了引理1中 d 与 a 平行的情形。

(C) 圆柱侧面任意点处的边界值：即对 $h \in [0, H_T]$ 固定检查 θ 的端点或解方程 $\partial_\theta \Psi = 0$ 。

(D) 圆柱体顶/底面圆心方向极值：直接计算距离 $|P - C|$ 的最大值。

对上述候选点集合计算每一点的 $d_{min}^2(P; t)$ 取最大值，若其小于等于 R_s^2 ，则判定为完全遮蔽；否则不完全遮蔽。

Step7 候选点构造的具体代数方程

对于求解圆柱顶/底的最远点，我们设 C_{xy} 与 $T_{0,xy}$ 的水平距离为

$$d_{xy} = |C_{xy} - T_{0,xy}|$$

则圆柱体上到C的最大水平距离出现在侧面外切方向，最大点的平方距离为

$$D_{\max,xy} = (d_{xy} + R_T)^2$$

再结合竖直高度差，顶或底的最远点取决于z距离的更大端点，则得出距离公式：

$$\max_{P \in T} |P - C|^2 = (d_{xy} + R_T)^2 + \max\{(z_C - 0)^2, (z_C - H_T)^2\}$$

对于求解直线与圆柱面交点方程（ d 与 a 平行的情形），我们取 xy 平面分量，则直线 $L(\lambda) = M + \lambda a$ 与圆柱侧面交点满足，

$$|(M_{xy} + \lambda a_{xy}) - T_{0,xy}|^2 = R_T^2, \quad z \text{ 坐标约束为 } 0 \leq M_z + \lambda a_z \leq H_T$$

展开后得到关于 λ 的二次方程：

$$|a_{xy}|^2 \lambda^2 + 2(M_{xy} - T_{0,xy}) \cdot a_{xy} \lambda + |M_{xy} - T_{0,xy}|^2 - R_T^2 = 0$$

求出实根 λ_k 并检查对应 z 是否在 $[0, H_T]$ 内且对应的 s^* 是否满足 $0 < s^* < 1$ ，每个合格的 λ_k 对应一个候选点 $P = M + \lambda_k a$ 。

对于求解圆柱侧面上 θ 的内点临界方程，我们固定 $h \in (0, H_T)$ 后对 Ψ 关于 θ 求导得到一维方程：

$$\frac{\partial}{\partial \theta} \Psi(\theta, h) = 0$$

把 $P(\theta, h)$ 与 d 代入分段表达：若 $s^*(\theta, h) \geq 1$ ，此方程变为 $\partial_\theta |P - C|^2 = 0$ ，可解析得到 θ 对应方向为 P_{xy} ，其指向 C_{xy} 的相反方向；若 $0 < s^* < 1$ ，则需对表达式 $|a|^2 - (a \cdot d)^2 / |d|^2$ 求导并解方程，得到可用数值方法求解的一维方程。

5.1.2 模型计算结果

基于上述模型，我们给出如下计算流程图：

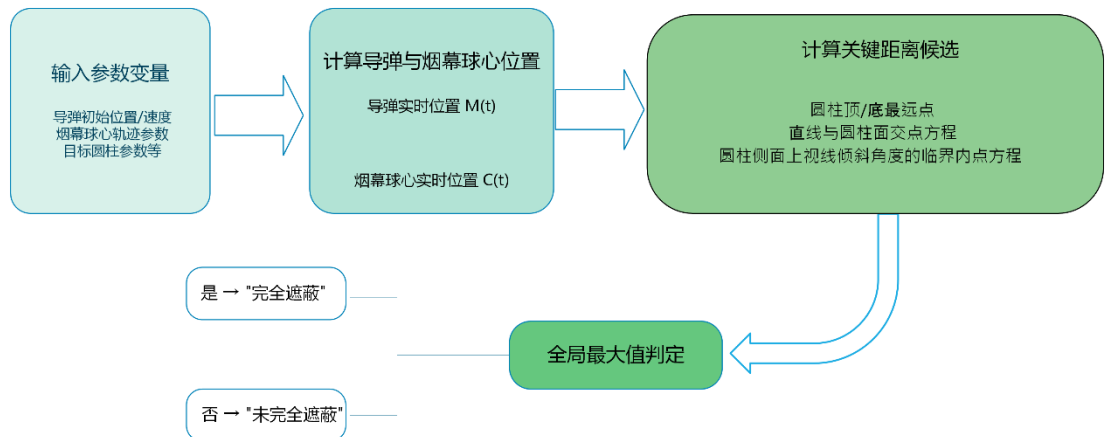


图 3：程序计算流程图

将数据代入距离公式，通过程序得出如下结果：完全遮蔽时段（单位：秒）为[8.041797, 9.448047]，完全遮蔽总时长为 **1.406250 s**。

此外，我们给出烟幕云团恰好开始完全遮蔽真目标的初始时刻左右，云团与导弹的三维空间图例：

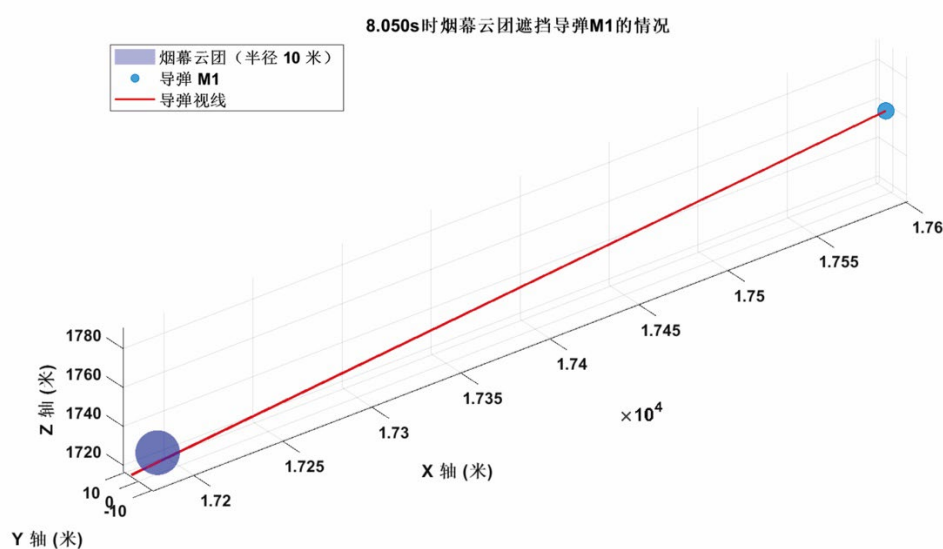


图 4：8.050s 时烟幕云团遮挡导弹 M1 的空间三维图

以及我们给出烟幕云团在有效范围内恰好完全包裹导弹的某一时刻的空间三维图例：

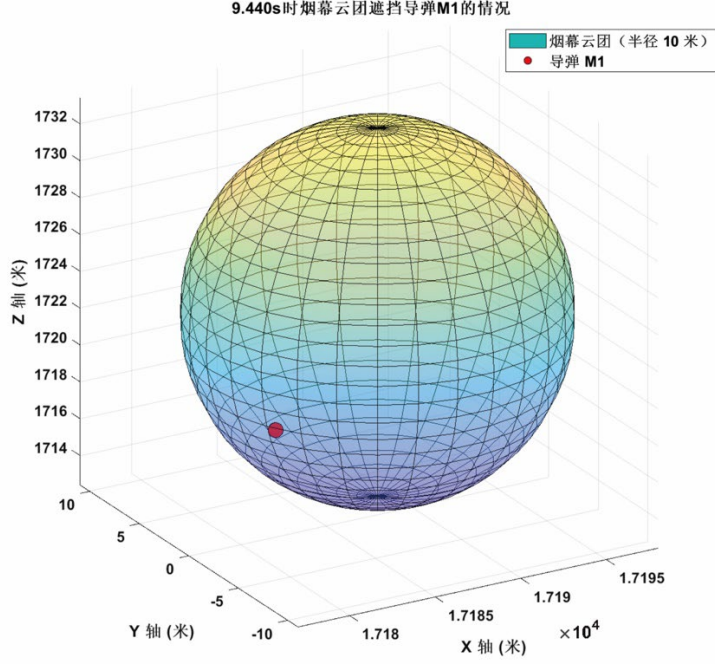


图 5: 9.440s 时烟幕云团遮挡导弹 M1 的空间三维图

5.2 问题二的求解

5.2.1 模型的建立

我们的核心任务是确定无人机 FY1 的航向、速度、投放点与起爆点（含时刻），使真目标（圆柱）被完全遮蔽的总时长最大。

我们先引入一些新的符号及定义。记无人机 FY1 速度为 $v_U \in [70, 140]$ ，记 $\widehat{u}_U(\psi) = (\cos \psi, \sin \psi, 0)$ ，则无人机位置函数为：

$$U(t) = U_0 + v_U \widehat{u}_U(\psi)t$$

定义在时刻 t ，对任意 $P \in \mathcal{T}$ ，导弹至 P 的视线段：

$$L(s; t, P) = M(t) + s(P - M(t)), \quad s \in [0, 1]$$

定义“最不利点”包络函数：

$$\Phi(t) = \max_{P \in \mathcal{T}} d_{min}^2(P; t)$$

定义遮蔽指示函数：

$$\chi(t) = 1_{[t_e, t_e + T_s]}(t) \cdot 1_{[0, t_{imp}]}(t) \cdot 1\{\Phi(t) \leq R_s^2\}$$

Step1: 确定决策变量、约束条件与优化目标

我们记 $x = (\psi, v_U, t_e, \Delta)$ 为决策向量，其中 $t_{rel} = t_e - \Delta \geq 0, 70 \leq v_U \leq 140$ 。

约束条件有如下四条：

- (1) $\widehat{u}_U \cdot e_z = 0$ ，保证无人机等高度飞行。
- (2) 云团有效窗为 $[t_e, t_e + T_s]$ 。

(3) 导弹在空窗为 $[0, t_{\text{imp}}]$ 。

(4) 云团中心 z 轴坐标 $C_z(t) \geq 0, \forall t \in [t_e, t_e + T_s]$

为使完全遮蔽总时长最大化，我们确定优化目标如下：

$$J(x) = \int_0^{t_{\text{imp}}} \chi(t) dt \rightarrow \max$$

显然，

$$J(x) \leq \min\{T_s, (t_{\text{imp}} - t_e)_+\}$$

Step2: 得出起爆时刻与延迟的决策方程

设最优方案在有效窗内形成一个或多个不相交遮蔽区间 $[t_{a,j}, t_{b,j}]$ 。若某个区间不被外部窗口端点截断（即非 t_e 或 $t_e + T_s$ ），则该区间两端满足

$$\Phi(t_a) = R_s^2, \quad \Phi(t_b) = R_s^2, \quad \Phi_t(t_a) > 0, \quad \Phi_t(t_b) < 0$$

对时间类参数 $\theta \in \{t_e, \Delta\}$ ，由隐函数求导得

$$\frac{dt_a}{d\theta} = -\frac{\Phi_\theta}{\Phi_t}|_{t_a}, \quad \frac{dt_b}{d\theta} = -\frac{\Phi_\theta}{\Phi_t}|_{t_b}$$

区间长度 $J_{ab} = t_b - t_a$ 在内点最优需满足

$$\frac{\Phi_\theta}{\Phi_t}|_{t_a} = \frac{\Phi_\theta}{\Phi_t}|_{t_b} \quad (\theta = t_e, \Delta)$$

为计算上述偏导，记单点主导函数 $f(\mathbf{P}, t) = d_{\min}^2(\mathbf{P}; t)$ 。由包络定理， Φ 在“主动点”处等于 f 。定义关于云团中心的梯度

$$G_C := \frac{\partial f}{\partial C} = \begin{cases} -2a, s^* \leq 0 \\ -2\left(a - \frac{(a \cdot d)}{|d|^2}d\right), 0 < s^* < 1 \\ -2(P - C), s^* \geq 1 \end{cases}$$

并有

$$\dot{M} = v_M \widehat{u_M}, \quad \dot{C} = (0, 0, -v_s)。$$

于是

$$\Phi_t = \frac{\partial f}{\partial M} \cdot \dot{M} + G_C \cdot \dot{C}$$

$$\Phi_{t_e} = G_C \cdot (v_U \widehat{u_U}(\psi) + (0, 0, v_s)), \quad \Phi_\Delta = G_C \cdot (0, 0, -g\Delta)$$

该“端点—灵敏度”必要条件给出起爆时刻与延迟的决策方程，用于解析校准与数值微调，使遮蔽区间在两端“平衡地拉伸”。

由于 $\Phi(t)$ 是不同几何主导点的上包络，且线段与球最近点的分段形态会切换，故在 $[t_e, t_e + T_s] \cap [0, t_{imp}]$ 内可能出现多个遮蔽区间 $\{[t_{a,j}, t_{b,j}]\}$ ，则总目标为：

$$J(x) = \sum_j (t_{b,j} - t_{a,j})$$

5.2.2 模型计算结果

我们给出数值求解框架如下：

- (1) 水平—竖直解耦启：先定 (ψ, v_U) 粗搜“水平落点风格”，再以 t_e 调整水平位置，用 Δ 调整起爆高度。
- (2) 时间集合构造：在有效窗内用**自适应步进扫描** $f(t) = \Phi(t) - R_s^2$ ，在符号变化处用**二分法**精化端点；若有多区间则取并集长度。
- (3) 层级优化：外层 (ψ, v_U) 粗搜；中层 t_e 粗搜且局部一维优化；内层 Δ 用**黄金分割**最大化区间总长度。
- (4) KKT 检查：若遮蔽端点贴合云窗边界 ($t_a = t_e$ 或 $t_b = t_e + T_s$)，采用相应的边界条件与互补性判断。

具体的程序流程图如下：

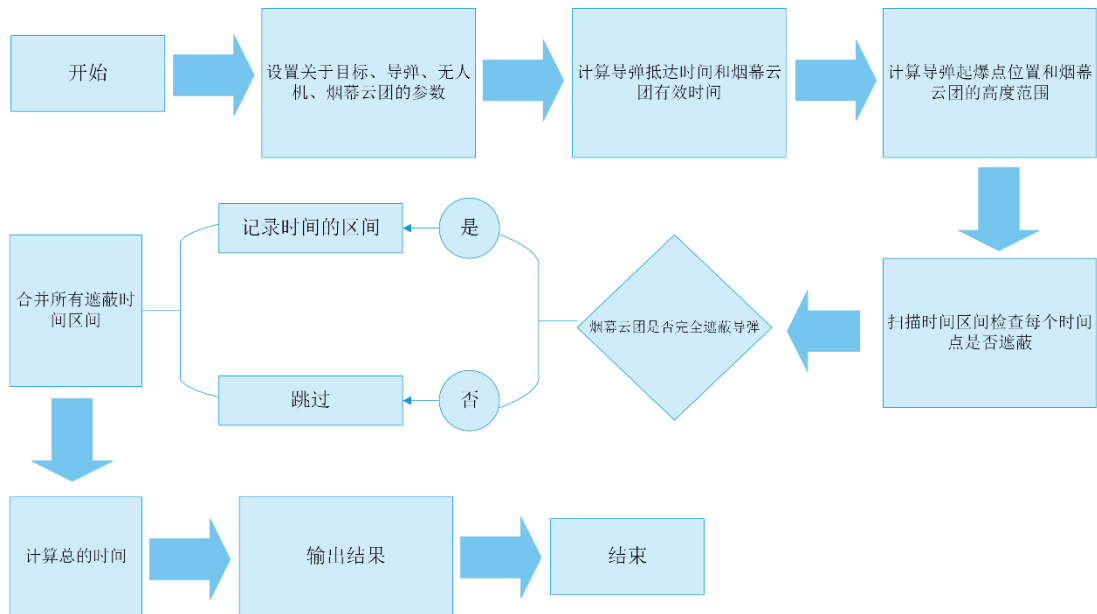


图 6：问题二的程序流程图

经过计算我们得出具体数据如下：

表 2：问题二结果数据

总遮蔽 时长 (s)	航向角 ψ (弧 度)	FY1 速度 v_U (m/ s)	起爆时 刻 t_e (s)	引信延 迟 Δ (s)	释放时 刻 t_{rel} (s)	起爆点 C_0	遮蔽区 间
4.5853	0.1309	110.0	0.856	0.851	0.004	(17893.354, 12.290, 1796.448)	[0.8642, 5.4495]

5.3 问题三的求解

5.3.1 模型的建立

我们延续使用前两问中的符号。受限篇幅下面的运动学方程我们只考虑 3 枚干扰弹，而导弹与无人机的运动学方程与前述问题一致。另外我们重新给出遮蔽判据。

Step1 运动学方程再建立

我们记第*i*枚干扰弹的释放时刻为 $t_{r,i}$ ，引信延迟为 Δ_i ，起爆时刻为

$$t_{e,i} = t_{r,i} + \Delta_i$$

释放点为

$$G_i = U(t_{r,i}) = U_0 + v_U \hat{u}(\psi) t_{r,i}$$

起爆点为

$$C_i^0 = (x_{e,i}, y_{e,i}, z_{e,i})$$

其中 $x_{e,i} = G_{i,x} + v_U \cos \psi$ ， $y_{e,i} = G_{i,y} + v_U \sin \psi$ ， $z_{e,i} = G_{i,z} - \frac{1}{2} g \Delta_i^2$

爆炸后云团中心下沉的位置函数为

$$C_i(t) = C_i^0 + (0, 0, -v_s(t - t_{e,i})), \quad t \geq t_{e,i}$$

要保证末端不触地或不低于阈值 z_{min} ，则由

$$C_{i,z}(t_{e,i} + T_s) = z_{e,i} - v_s T_s \geq z_{min}$$

可以得到

$$0 \leq \Delta_i \leq \overline{\Delta_i} \equiv \sqrt{\max\left\{0, \frac{2(G_{i,z} - z_{min} - v_s T_s)}{g}\right\}}$$

Step2 几何遮蔽判定

对任一点 Q （来自目标侧面与上下底面离散网格），导弹与 Q 间的视线段记为

$$l_Q(t) = \{M(t) + s(Q - M(t)) \mid s \in [0,1]\}$$

根据问题二的讨论我们可以类似得出第*i*枚云团中心到线段的最短距离平方为

$$\text{dist}^2(C_i(t), l_Q(t)) = \begin{cases} |C_i(t) - M(t)|^2, s^* \leq 0 \\ |C_i(t) - Q|^2, s^* \geq 1 \\ |C_i(t) - M(t)|^2 - \frac{((C_i(t) - M(t)) \cdot (Q - M(t)))^2}{|Q - M(t)|^2}, s^* \in (0,1) \end{cases}$$

其中

$$s^* = \frac{(C_i(t) - M(t)) \cdot (Q - M(t))}{|Q - M(t)|^2}$$

每一枚干扰弹的判据函数为

$$\Phi_i(t) \equiv \max_{Q \in Q_{\text{null}}} \text{dist}^2(C_i(t), l_Q(t))$$

若存在任一枚使真目标所有点被罩，则视为完全遮蔽，即

$$\Phi_{\min}(t) \equiv \min_{i=1,2,3} \Phi_i(t) \leq R_s^2$$

单枚有效时间窗为

$$t \in [t_{L,i}, t_{R,i}], \text{ 其中 } t_{L,i} = \max(0, t_{e,i}), t_{R,i} = \min(t_{imp}, t_{e,i} + T_s)$$

Step3 决策优化与条件约束

决策变量包含 (ψ, v_U) 及三枚干扰弹的 $(t_{e,i}, \Delta_i)_{i=1,2,3}$ 。记三枚弹单独得到的遮蔽区间并为 $\mathcal{I}_i$ ，联合并集长度为 $\mathcal{L} = \text{length}(\cup_{i=1}^3 \mathcal{I}_i)$ 。需要满足的释放间隔约束：

$$t_{r,i} = t_{e,i} - \Delta_i \geq 0, t_{r,j} - t_{r,i} \geq S (\forall i < j), \text{ 其中 } S \geq 1$$

我们要在满足 $v_U \in [v_U^{\min}, v_U^{\max}]$ ， $0 \leq \Delta_i \leq \overline{\Delta_i}$ ，及上述约束条件下找到

$$\max \mathcal{L}(\psi, v_U, \{t_{e,i}, \Delta_i\}_{i=1}^3)$$

5.3.2 模型的求解

我们采用“外层全局采样 + 中层组合选择 + 内层连续微调”的多层启发式框架，并配以若干工程性加速与修复机制。整体流程如下：

Step1 外层： (ψ, v_U) 候选生成与筛选

我们先使用低差异序列在给定域内对 (ψ, v_U) 做准均匀采样，得到一组姿态—速度模式点。对每个模式点独立构造“单弹候选库”，并在覆盖最大化意义上从库中选三枚，得到一个三弹方案，记为“种子”。然后以种子为起点做模式搜索：在 ψ 与 v_U 的小邻域内试探若干方向步长，并在每次试探后对三弹做局部连续微调，

选择提升目标值的方向，逐轮缩小步长直至收敛。最后对多个种子并行或串行竞争，保留最优解。

Step2 中层：单弹候选库的构造与三枚选择

在已定 (ψ, v_U) 下，先对 (t_e, Δ) 做两路采样：

(1) 基础网格：在 $t_e \in [0, t_{imp}]$ 均匀取点；在每个 t_e 上对 $\Delta \in [0, \min\{t_e, \overline{\Delta}(t_e)\}]$ 取等距样点。

(2) 几何引导：依据导弹在平面上的轨迹，通过投影几何求交，推导出一组“更可能产生遮蔽”的 t_e 候选，在这些 t_e 处对 Δ 加密采样。

然后对每对 (t_e, Δ) 计算单弹遮蔽区间 $\mathcal{I}(t_e, \Delta)$ 及长度；在每个 t_e -桶内保留若干 top-k 候选；再做去重（区间图样+释放时刻的量化签名）与 τ -分桶（按释放时刻分段留存），形成规模受控、分布均匀的候选库。

最后三枚选择采用覆盖最大化的组合启发式：

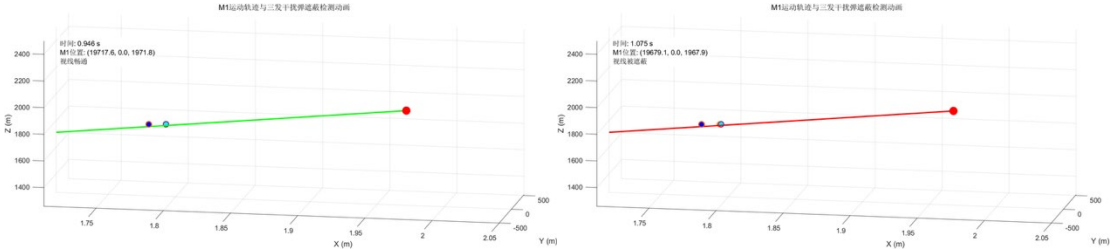
- (A) 将时间轴以步长 δt 离散为网格 $\{t_g\}$ ，把每个候选 $c \in \mathcal{C}$ 的区间集合 $\mathcal{I}_c$ 映射为布尔覆盖向量 m_c ；
- (B) 在最小间隔约束 $|\tau_i - \tau_j| \geq s$ 下进行贪心选取，从空集出发，反复选取在满足间隔约束的前提下，使 $|\bigvee m|$ 增加最多的候选，直至选满三枚或无法增加；
- (C) 若不足三枚，则对强候选尝试 τ 平移（等价于在固定 t_e 下调整 Δ ），只要满足间隔约束且覆盖变好即纳入；
- (D) 在三枚已定的情况下，做“1-out-1-in”交换搜索，只要替换后网格覆盖计数增加就接受，直到无改进为止[1]。

Step3 内层：连续微调（黄金分割 + 交替优化）

在固定 (ψ, v_U) 和三枚弹的身份后，交替对每枚弹的 Δ_i 与 $t_{e,i}$ 做一维黄金分割搜索：对于“ Δ -步”，在 $[0, \overline{\Delta}(t_e)]$ 上最大化联合长度，试探点一律做间隔可行性检查（释放时刻集合 $\{\tau_i\}$ 两两差值 $\geq s$ ）与（若启用）云团末端高度检查，若违反则该方向打分置为极小；对于“ t_e -步”，在 $[t_e - \omega, t_e + \omega] \cap [0, t_{imp}]$ 的局部窗口内最大化联合长度，同样受间隔与物理边界约束。交替两轮后，触发一次间隔投影修复：先将三弹按 τ 升序排序；再逐个检查相邻差值，如不足 s ，优先顺延 t_e 补足差额；若 t_e 已触上限，则减少 Δ （下界 0）来增大 τ ；最后还原原顺序并校验所有物理边界。随后统一复评估并集长度，得到当前局部最优。

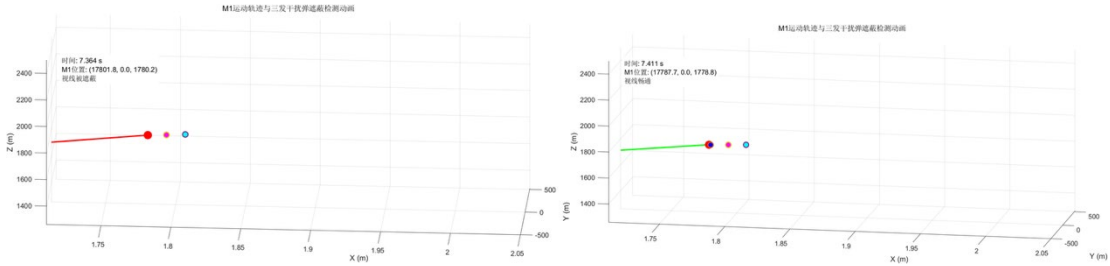
5.3.3 模型计算结果

我们绘制了 M1 运动轨迹与三发干扰弹遮蔽检测动图，并截取其中四个具有代表性的时刻：即将开始遮蔽，开始遮蔽，即将结束遮蔽以及结束遮蔽时刻。图例如下：



子图 1： 0.946s时干扰弹位置图

子图 2： 1.075s时干扰弹位置图



子图 3： 7.364s时干扰弹位置图

子图 4： 7.411s时干扰弹位置图

图 7： M1 运动轨迹与三发干扰弹位置图示

经过计算我们得到如下数据：

表 3： 问题三结果数据

无人 机运 动方 向	无 人 机 运 动 速 度 (m/ s))	烟 幕 干 扰 弹 编 号	投 放 点 坐 标	起 爆 点 坐 标	有 效 干 扰 时 长 (s))	遮 蔽 区 间
179.35 63	11 0	1	(17800,0,1800)	(17437.02, 4.078394, 1746.639)	3.2251 28	[4.334821, 7.559949]
		2	(17690.01,1.235877 ,1800)	(17283.03,5.808622,17 32.919)	4.3232 73	[4.7, 9.023273]
		3	(17580.01,2.471754 , 1800)	(17129.04, 7.53885,1717.631)	3.5296 33	[6.1,9.629 633]

5.4 问题四的求解

5.4.1 模型的建立

我们沿用问题一、二、三的记号，一般情况的运动学方程及遮蔽判据模型与前述问题一致。我们现先给出无人机能在导弹抵达目标前将干扰弹投掷到真目标上顶面圆心情形的运动学方程。

记目标点圆柱顶面中心点 $G = (x_G, y_G, z_G) = (0, 200, 10)$ ，无人机初始位置 $P_0 = (x_0, y_0, z_0)$ 、速度 v ，起爆瞬时水平位置 (x_G, y_G) ，起爆高度 $z_b = 10$ ，最终时刻 $t_{\text{final}} = t_b$ ，起爆时刻为

$$t_b^* = t_{\text{arr}} = \frac{|(x_G - x_0, y_G - y_0)|}{v}$$

设水平单位航向向量

$$\mathbf{u}_{xy} = \frac{(x_G - x_0, y_G - y_0)}{\sqrt{(x_G - x_0)^2 + (y_G - y_0)^2}}, \quad \theta = \text{atan2}(y_G - y_0, x_G - x_0)$$

故下面我们可以得出关键运动学公式：

(1) 无人机最早到达目标上空时间

$$t_{\text{arr}} = \frac{d}{v}, \quad d = \sqrt{(x_G - x_0)^2 + (y_G - y_0)^2}$$

(2) 延时引信（投放后自由落体到 z_b 用时 τ ）

$$\tau^* = \sqrt{\frac{2(z_0 - z_b)}{g}} = \sqrt{\frac{2(z_0 - 10)}{g}}$$

(3) 最早投放时刻

$$t_{\text{rel}}^* = t_b^* - \tau^* = \frac{d}{v} - \sqrt{\frac{2(z_0 - 10)}{g}}$$

(4) 投放坐标

$$B = (x_b, y_b, z_b) = (x_G, y_G, 10)$$

(5) 起爆坐标

$$D = (x_d, y_d, z_d) = (x_0, y_0, z_0) + v t_{\text{rel}}^* (\mathbf{u}_{xy}, 0)$$

随后针对本问在问题二基础上优化总目标。我们记云团有效窗为

$$I_k^{\text{adm}} = [t_{e,k}, \min\{t_{\text{imp}}, t_{e,k} + T_s, t_{e,k} + C_{0,k}^{(z)}/v_s\}]$$

三架无人机各投一枚干扰弹（参数分别为 $\theta_k = (\psi_k, v_k, t_{e,k}, \Delta_k)$ ），目标是在

$[0, t_{\text{imp}}]$ 上最大化三枚云团遮蔽集合的并集时长：

$$\mathcal{A}_k = \{t \in [0, t_{\text{imp}}]: \Phi_k(t) \leq R_s^2\}, \quad \mathcal{U}(\Theta) = \mathcal{A}_1 \cup \mathcal{A}_2 \cup \mathcal{A}_3$$

$$\max_{\Theta = (\theta_1, \theta_2, \theta_3)} \mu(\mathcal{U}(\Theta)) = \mu\left(\bigcup_{k=1}^3 \mathcal{I}_k(\theta_k)\right)$$

其中 $\mu(\cdot)$ 为实线上的勒贝格测度（即总时长）；约束条件如下：

$$v_{\min} \leq v_k \leq v_{\max}, \quad \psi_k \in [-\pi, \pi), \quad 0 \leq t_{e,k} \leq t_{\text{imp}}$$

$$\Delta_k \geq 0, \quad \tau_k = t_{e,k} - \Delta_k \geq 0, \quad C_k^{(z)}(t) \geq 0, \quad \forall t \in I_k^{\text{adm}}$$

5.4.2 模型的求解

经过计算我们发现 FY3 符合上述模型建立时的特殊情形，则 FY3 水平面航线朝向真目标即可，故下面的模型求解我们只要考虑 FY1 及 FY2。

由于 Θ 维数高、目标非凸且对离散网格敏感，我们采用“结构化启发式 + 连续一维精化”的混合求解框架，核心步骤如下：

Step1 单机候选生成

(1) 航向/速度外层网格：对 FY k ($k=1, 2$)，我们以“指向原点”的航向

$$\psi_{0,k} = \text{atan2}(-U_{0,k}^{(y)}, -U_{0,k}^{(x)})$$

为中心，取对称角域 $[\psi_{0,k} - \Delta_\psi, \psi_{0,k} + \Delta_\psi]$ 等步分割；速度 v_k 在 $[v_{\min}, v_{\max}]$ 等距采样。对“ ψ 接近 $\psi_{0,k}$ ”的角域，特别加密早期 $t_{e,k} \in [0, t_{\text{dense}}]$ 的网格（步长如0.04 s），同时对末段 $[t_{\text{imp}} - T_s, t_{\text{imp}}]$ 也加密；其余区间中等密度。

(2) Δ 两阶段：对每个固定 $t_{e,k}$ ，先进性粗采样：在 $[0, \Delta_{\max}(t_{e,k})]$ 上取少量等距点，若出现任一遮蔽（即 $L_k > 0$ ），则认为该 $t_{e,k}$ 有潜力；后进行黄金分割：仅对“有遮蔽”的 $t_{e,k}$ 进行连续一维优化

$$\Delta_k^* = \arg \max_{\Delta \in [0, \Delta_{\max}]} L_k(\psi_k, v_k, t_{e,k}, \Delta)$$

并返回对应的 $\mathcal{J}_k$ 。

(3) 按 t_e 桶保留 + 全局截断：为保证多样性，对 $t_{e,k}$ 做分桶（如0.25s一桶），每桶留 Top- p ；再全局留 Top- P 。

(4) 二次精修：对候选 $(t_{e,k}, \Delta_k)$ 再做“ $t_{e \text{ 黄金}} \circ \Delta \text{ 黄金}$ ”，以消除离散网格偏差。

经以上步骤最终返回每机的候选集合 $\mathcal{C}_k = \{\theta_k^{(m)}\}_{m=1}^{N_k}$ 与其区间族 $\mathcal{J}_k^{(m)}$ 。

Step2 三机组合

定义区间并集长度算子为 $\mathcal{L}(\cdot)$ 。下面我们采用轻量而稳定的三步：

(1) 贪心初始化：先取 $k = 1$ 的最佳单机

$$m^* = \arg \max_m \mathcal{L}(\mathcal{J}_1^{(m)});$$

再从 $\mathcal{C}_2$ 中选 n^* 使 $\mathcal{L}(\mathcal{J}_1^{(m^*)} \cup \mathcal{J}_2^{(n^*)})$ 最大。

(2) LNS 局部邻域搜索：对当前两枚分别在 $\pm\delta t_e/\pm\delta$ 的小邻域内重新计算区间。

为直观展现模型求解流程，我们给出流程图：

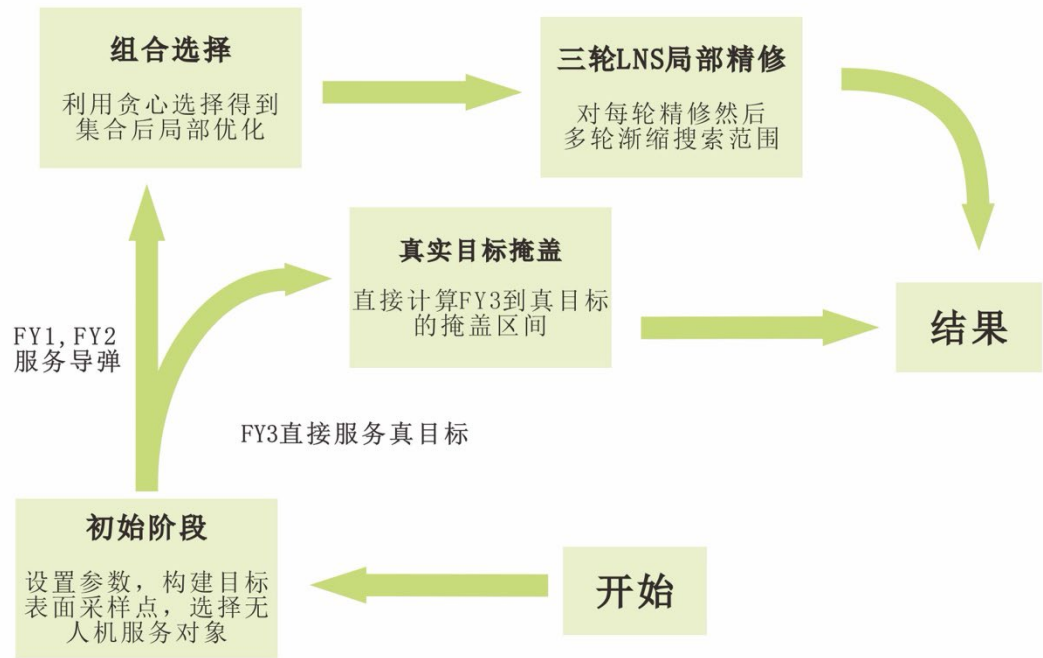


图 8：问题四模型求解流程图

5. 4. 3 模型的计算结果

经过计算我们得到如下数据：

表 4： 问题四结果数据

无人 机编 号	无人 机运 动方 向	无人 机运 动速 度 (m/s)	投放点的坐 标	起爆点的坐 标	有效干 扰时长 (s)	有效遮蔽区间
FY 1	7.500	110	(17800.545 , 0.072, 1800)	(17893.354 , 12.290, 1796.448)	4.5853	[0.8642, 5.4495]
FY 2	320.62 5	130	(13469.586 , 193.942, 1400)	(13694.184 , 9.619, 1375.498)	3.9214	[16.8623, 20.7837]

FY 3	151.92 8	140	(1466.246, -581.997, 700)	(0, 200, 10)	18.428 5	[48.5714,66.999 9]
---------	-------------	-----	---------------------------------	-----------------	-------------	-----------------------

5.5 问题五的求解

由于符号新增较多，我们重新给出符号说明及运动学方程。

5.5.1 模型符号说明

(1) 导弹：导弹集合 $\mathcal{I} = \{1,2,3\}$ ，初始位置 $M_i(0)$ 已知，恒速直线指向假目标 $O = (0,0,0)$ ： $M_i(t) = M_i(0) + v_M t u_i$, $u_i = \frac{O - M_i(0)}{|O - M_i(0)|}$, $v_M = 300$ 。

(2) 无人机 (UAV)：初始位置 $p_j^0 = (x_j^0, y_j^0, h_j)$ 已知，位置函数如下：

$$P_j(t) = p_j^0 + v_j t u_j, \quad |u_j| = 1, \quad v_j \in [70, 140]$$

每架无人机相邻投放干扰弹时刻 $|r_{j,k+1} - r_{j,k}| \geq \Delta r_{min} = 1$ 。

(3) 干扰弹：第 k 枚弹投放时刻 $r_{j,k}$ ，延迟引信 $\Delta_{j,k}$ ，起爆时刻 $e_{j,k} = r_{j,k} + \Delta_{j,k}$ ，起爆点位置

$$B_{j,k} = \underbrace{P_j(r_{j,k}) + v_j u_j \Delta_{j,k}}_{\text{水平位移}} + \underbrace{\left(0, 0, -\frac{1}{2} g \Delta_{j,k}^2\right)}_{\text{竖直下落}}$$

(4) 第 k 个云团中心位置：

$$C_{j,k}(t) = B_{j,k} - v_s(t - e_{j,k}) e_z, \quad t \in [e_{j,k}, e_{j,k} + T_s]$$

5.5.2 模型的建立

由于球态完全遮蔽判据模型与问题三一致，这里我们只给出触地阶段的完全遮蔽判据。

Step1 建立触地阶段的完全遮蔽判据模型

由模型假设云团触地后，用以 B^{xy} 为中心、边长 $10m$ 的立方体近似云团：

$$\mathcal{B} = \{x \in R^3: b_{min} \leq x \leq b_{max}\},$$

其中 $b_{min} = \left(B_x - \frac{L}{2}, B_y - \frac{L}{2}, 0\right)$, $b_{max} = \left(B_x + \frac{L}{2}, B_y + \frac{L}{2}, H_c\right)$, $L = 10, H_c = 10$

完全遮蔽判据如下：

$$\forall Q \in \mathcal{T}: \overline{M_i(t)Q} \cap \mathcal{B} \neq \emptyset$$

Step2 建立航线选择决策模型

记变量 $x_{j,s} \in \{0,1\}$, $s \in \{T, M_1, M_2, M_3\}$ ，偏好权重 $\beta_{j,s} \geq 0$ (未偏好时取 1)，

本文要在约束条件

$$\sum_s x_{j,s} = 1, \forall j, \quad \sum_j x_{j,M_i} \leq 1, i = 1, 2, 3, \quad \sum_j x_{j,T} \geq 1; \quad x_{j,s} \in \{0, 1\}$$

下找到

$$\min \sum_j \sum_s \frac{\tau_{j,s}}{\beta_{j,s}} x_{j,s}$$

其中 $\tau_{j,s}$ 为可达时间下界，记导弹视线走廊 $l_i = \overline{m_i^{xy} O^{xy}}$ ，则

$$\tau_{j,i} = \frac{d(p_j^{xy}, l_i)}{V_{max}}$$

上述给出的最小值仅用于方向性分配，并在下一阶段作为“软偏好”注入。

Step3 建立多弹时序最大覆盖模型

为可解性，本文引入有限候选集离散化：对每架无人机 j ，给定一个航向—速度候选集合 $\mathcal{H}_j = \{(u_{j,m}, v_{j,m})\}_m$ ；对每个 $(u_{j,m}, v_{j,m})$ ，记离散投放—延迟集合 $\mathcal{E}_{j,m} = \{(r, \Delta)\}$ ；记事件形成 $e = (j, m, r, \Delta)$ ；导弹 i 在离散时间网格 $\mathcal{T} = \{t_l\}$ 上的完全遮蔽指示为

$$a_{e,i,l} = \begin{cases} 1, & \text{若 } t_l \in [e_e, e_e + T_s] \text{ 且 (球/立方体) 完全遮蔽成立} \\ 0, & \text{反之} \end{cases}$$

下面我们确定决策变量与约束条件。选取事件二元变量 $x_e \in \{0, 1\}$ ；每机至多选一个 m ，航向—速度锁定如下：

$$\sum_m h_{j,m} = 1, \quad h_{j,m} \in \{0, 1\}; \quad x_e \leq h_{j,m_e}$$

同机容量与时间的冲突（构造冲突图 $\mathcal{C}_j$ ：同机且 $|r_e - r_{e'}| < \Delta_{r_{\min}}$ ）：

$$\sum_{e \in \mathcal{E}_j} x_e \leq 3, \quad x_e + x_{e'} \leq 1, \forall (e, e') \in \mathcal{C}_j$$

记覆盖线性化变量 $z_{i,l} \in [0, 1]$ ，需满足

$$z_{i,l} \leq \sum_e a_{e,i,l} x_e, \quad z_{i,l} \leq 1$$

其表示导弹 i 在时间片 t_l 被至少一个事件完全遮蔽。

由最大联合覆盖，我们得出目标函数

$$\max J_{\Delta t} = \sum_{i \in \mathcal{I}} \sum_1 w_{i,l} z_{i,l} \Delta t$$

其中 Δt 为时间网格步长，早期无加权时 $w_{i,l}$ 取值为 1。随后将分配软偏好与几何一致性奖励合并到 $a_{e,i,l}$ ，得到新的“有效权”：

$$\widetilde{a}_{e,i,l} = \underbrace{\lambda_{j,i}}_{\text{分配权重 } (>1 \text{ 若 } j \rightarrow M_i)} \cdot \underbrace{\kappa_e}_{\text{几何一致性 } \left(\kappa_e = 1 + \alpha e^{-\frac{d_e^2}{\sigma^2}} \right)} \cdot a_{e,i,l}$$

其中 d_e 为起爆点到被指派对象（真目标轴心或某导弹走廊）的水平距离。

5.5.2 模型的求解

Step1 求解初步任务分配

计算 $\tau_{j,T}, \tau_{j,i}$ ，再结合业务规则得到无人机的初次任务分配 ($j \mapsto s$)，并将结果转化为 $\lambda_{j,i} > 1$ 与几何一致性函数 κ_e [2]。

Step2 候选生成与粗判

对每个无人机构造 $\mathcal{H}_j$ ，并离散 $\mathcal{E}_{j,m}$ 。随后对每个事件 e ，先计算 (e_e, B_e) （若 $(B_e)_z < 0$ 或 $e_e > T$ 则丢弃）；然后在稀疏 $\partial \mathcal{T}$ 网格与稀疏时间上按球态判据粗筛可能有效的候选，显著降维。

Step3 最大覆盖的近似选择

将模型建立中的目标函数视为带冲突约束的最大覆盖问题，加入约束后（每机 ≤ 3 、冲突图、航向/速度一致），近似落在 p -可扩展系统范畴，朴素贪心具常数因子保证。贪心框架如下：以“新增覆盖秒数”为边际增益；首次为某无人机选择事件时，锁定该无人机的 (u, v) ，剔除与之不一致的候选；维护同机容量与时间冲突；迭代至无正增益。随后进行局部搜索，对已选集合 S 在同一 (u, v) 的邻域内尝试以 e' 替换 e ，若 $F(S \setminus \{e\} \cup \{e'\})$ 提升则接受。若某无人机未发 3 发，则在不违反锁定与间隔的前提下，按“弥补未覆盖时间段”的增益再次贪心补入，直至满载或无正增益。

Step4 连续时间精算与区间合并

对每个事件 e 与导弹 i ，在 $[e_e, \min(e_e + T_s, T)]$ 内执行时间扫描，通过两种遮蔽判据模型得到一组区间 $\{[a, b]\}$ ，并按容差合并邻接短段。对固定导弹 i ，合并所有事件的区间并集，得到 $\mathcal{U}_i$ ，并计算

$$J = \sum_{i \in \mathcal{I}} \sum_{[a,b] \in \mathcal{U}_i} (b - a)$$

5.5.3 模型计算结果

表 5: 无人机分配结果

无人机	建议分配	估计最短到达时间下界(s)
FY1	拦截 M1	0
FY2	拦截 M2	7.29
FY3	真目标	48.57
FY4	拦截 M2	11.8
FY5	拦截 M3	11.18

表 6: 问题五结果数据

无人机编号	无人机运动方向	无人机运动速度(m/s)	烟幕干扰弹编号	投放点坐标	起爆点坐标	有效干扰时长(s)	干扰的导弹编号	有效遮蔽区间
FY1	179.3563	110	1	(17800, 0, 1800)	(17437.02, 4.078394, 1746.639)	3.225128	M1	[4.334821, 7.559949]
			2	(17690.01, 1.235877, 1800)	(17283.03, 5.808622, 1732.919)	4.323273		[4.7,9.023273]
			3	(17580.01,2.471754,1800)	(17129.04,7.53885,1717.631)	3.529633		[6.1,9.629633]
FY2	-139.289	120	1	(11363.27,852.1196,1400)	(10808.4,374.681,1217.671)	3.552521	M2	[18.56872,22.12124]
FY3	151.928	140	1	(1466.246, -581.997, 700)	(0, 200, 10)	18.4285	M1	[48.5714,66.9999]
FY4	-130.707	110	1	(10426.08,1332.909,1800)	(9558.015,323.9334,1082.591)	3.036285	M2	[26.43505,29.47134]
			2	(10282.59,1166.136,1800)	(9328.445,57.0969,933.239)	1.57254	M1	[23.3,24.87254]
FY5	120.3948	140	1	(12149.99,-550.9,1300)	(12057.91,-393.914,1291.719)	2.99057	M3	[16.95829,19.94886]
			2	(12220.83,-671.658,1300)	(12043.74,-369.762,1269.375)	0.88205	M3	[13.5,14.38205]

			3	(11724.99,173.6 503,1300)	(11604.57,378.939 5,1285.839)	3.286 469	M 2	[20.11696,2 3.40343]
--	--	--	---	------------------------------	----------------------------------	--------------	--------	-------------------------

经计算 M1、M2、M3 遮蔽总时长分别为 **25.2951s**、**23.0492s**、**15.66769s**。

此外我们给出对于三枚导弹各自的遮蔽时间区间示意图如下：

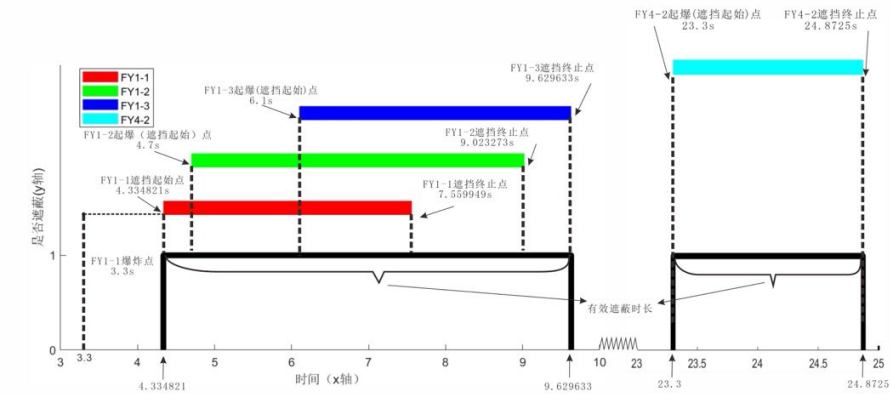


图 9：导弹 M1 遮蔽时间区间示意图

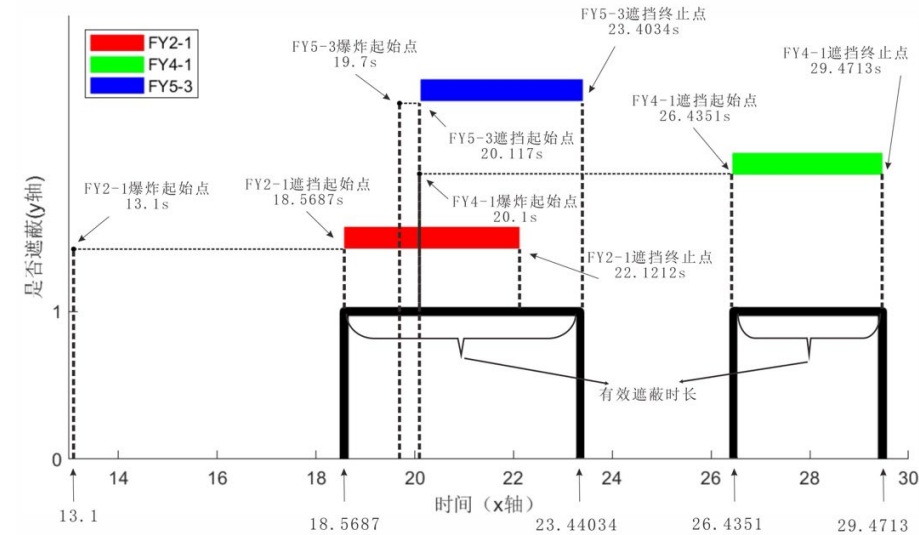


图 10：导弹 M2 遮蔽时间区间示意图

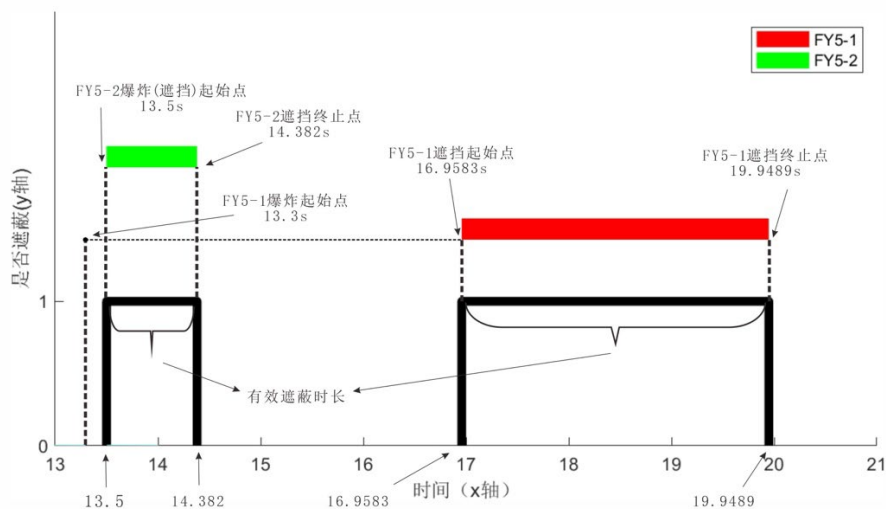


图 11：导弹 M3 遮蔽时间区间示意图

六、模型的评价及推广

6.1 模型评价

6.1.1 模型优点

(1) 在防御目标的选择上，能够根据不同情境做出灵活决策，既可通过烟幕直接阻挡导弹视线，又可通过优化烟幕布设策略，有效延长真目标的遮蔽时间。

(2) 在防御效果的提升上，本文提出的“末端覆盖协同防护模型”将无人机分工为“末端覆盖”与“前端协同”两个层次，使得烟幕干扰在时空维度上形成连续与互补，大幅度延长了目标在关键时段的防护时间。

(3) 计算上精度较传统单点遮蔽更高、判据更严谨，且能适应更复杂的多导弹、多无人机场景，体现出较强的鲁棒性与可扩展性。

(4) 模型有效揭示了烟幕遮蔽的几何本质与优化机制，具有较高的理论意义和工程应用价值。

6.1.2 模型缺点

(1) 在问题五中，FY3 仅被设计为承担末端覆盖任务，即在导弹接近目标的关键阶段实现完全遮蔽，忽略了 FY3 在前端提前布设烟幕的潜在价值。

(2) 部分无人机由于受到飞行时间与投放窗口的限制，未能完成三枚干扰弹的最优释放。

6.2 模型推广

(1) 模型若能在末端防护的同时引入其前期投放的可能性，便可以在更大范围上实现多层覆盖，从而提升整体方案的灵活性与冗余度。

（2）未来可以考虑在模型中引入更高效的全局调度算法，或采用滚动优化与动态规划的方法，使无人机能够在有限时间内完成最大化的投放任务。

（3）在现实意义上，当两个或多个烟幕在时空上部分重合时，其遮蔽效果可能远强于单个云团的覆盖。若能在模型中引入多烟幕协同判据与扩散模拟，将更贴近真实战场环境，进一步提升结果的可信度与适用性。

七、参考文献

参考文献

- [1] 周志奇. 基于群智能算法的无人机航迹规划研究[D]. 河北科技大学, 2024.
- [2] 李猛. 基于智能优化与 RRT 算法的无人机任务规划方法研究[D]. 南京航空航天大学, 2012.
- [3] DeepSeek, DeepSeek-V3, 深度求索 (DeepSeek), 2025-09-05、2025-09-06、2025-09-07

八、附录

附录一 支撑文件列表

表 7：支撑文件列表

文件	说明
2025A1. py	问题一代码
2025A2. py	问题二代码
2025A3. py	问题三代码
2025A4. py	问题四代码
2025A5. py	问题五代码
result1. xlsx	问题三数据
result2. xlsx	问题四数据
result3. xlsx	问题五数据
A1. gif	问题一动图
A3. gif	问题三动图
AI 工具使用说明. pdf	AI 工具使用说明

附录二 程序代码

本文采用 Python 进行编码

问题一代码（核心部分）

```
import numpy as np
from math import sqrt, pi
from typing import Tuple, List, Dict, Any
# ===== 工具函数 =====
def norm2(v: np.ndarray) -> float:
    """
    计算向量的平方范数（L2 范数的平方）
    Args:
        v: 输入向量[3]
    Returns:
        向量的平方范数
    """
    return float(np.dot(v, v))
def unit_vec(from_p: np.ndarray, to_p: np.ndarray) -> np.ndarray:
    """
    计算从点 from_p 指向点 to_p 的单位向量（三维）
    Args:
        from_p: 起点坐标
        to_p: 终点坐标
    Returns:
        单位方向向量
    """
```

```

v = to_p - from_p
n = np.linalg.norm(v)
return v / n if n > 0 else np.zeros(3)
def unit_vec_xy(from_p: np.ndarray, to_p: np.ndarray) -> np.ndarray:
    """
    计算从点 from_p 指向点 to_p 的 XY 平面单位向量（忽略 Z 坐标）

    Args:
        from_p: 起点坐标
        to_p: 终点坐标

    Returns:
        XY 平面单位方向向量
    """
    vx = to_p[0] - from_p[0]
    vy = to_p[1] - from_p[1]
    n = (vx**2 + vy**2) ** 0.5
    if n <= 1e-12:
        return np.array([0.0, 0.0, 0.0])
    return np.array([vx / n, vy / n, 0.0])
# 计算无人机飞向假目标的单位方向向量
u_FY1 = unit_vec_xy(FY1_0, F)
# ===== 运动学模型函数 =====
def missile_pos(t: float, M0: np.ndarray = M1_0, v: float = v_M, Fp: np.ndarray = F) -> np.ndarray:
    """
    计算导弹在时刻 t 的位置（沿直线飞向假目标）

    Args:
        t: 时间 (s)
        M0: 导弹初始位置
        v: 导弹速度
        Fp: 假目标位置

    Returns:
        导弹在时刻 t 的位置
    """
    u = unit_vec(M0, Fp)
    return M0 + v * u * t
def missile_imp_time(M0: np.ndarray = M1_0, v: float = v_M, Fp: np.ndarray = F) -> float:
    """
    计算导弹到达假目标的时间

    Args:
        M0: 导弹初始位置
        v: 导弹速度
        Fp: 假目标位置

    Returns:
        导弹到达时间 (s)
    """

```

```

"""
    return np.linalg.norm(M0 - Fp) / v
def uav_pos(t: float, U0: np.ndarray = FY1_0, vU: float = v_U, u: np.ndarray = u_FY1) -> np.ndarray:
    """
    计算无人机在时刻 t 的位置（水平飞行）
    Args:
        t: 时间 (s)
        U0: 无人机初始位置
        vU: 无人机速度
        u: 无人机飞行方向单位向量
    Returns:
        无人机在时刻 t 的位置
    """
    if t < t_rec:
        return U0.copy()
    horiz = U0[:2] + vU * u[:2] * (t - t_rec)
    return np.array([horiz[0], horiz[1], U0[2]])
def grenade_pos(t: float, t_rel_local: float = t_rel, U0: np.ndarray = FY1_0,
                vU: float = v_U, u: np.ndarray = u_FY1, consider_gravity: bool = True) -> np.ndarray:
    """
    计算手榴弹在时刻 t 的位置（考虑重力抛物线轨迹）
    Args:
        t: 时间 (s)
        t_rel_local: 手榴弹投放时间
        U0: 无人机初始位置
        vU: 无人机速度
        u: 无人机飞行方向单位向量
        consider_gravity: 是否考虑重力
    Returns:
        手榴弹在时刻 t 的位置
    """
    if t < t_rel_local - 1e-12:
        raise ValueError("t must be >= t_rel for grenade_pos")
    # 投放时刻的初始位置和速度
    G0 = uav_pos(t_rel_local, U0, vU, u)
    v0 = np.array([vU * u[0], vU * u[1], 0.0])
    dt = t - t_rel_local
    # 考虑重力的抛物线运动
    if consider_gravity:
        return G0 + v0 * dt + 0.5 * np.array([0.0, 0.0, -g]) * dt * dt
    else:
        return G0 + v0 * dt
def cloud_center_at(t: float, t_e_local: float = t_e, consider_validity: bool = True) -> np.ndarray:
    """

```

计算烟幕云团中心在时刻 t 的位置（匀速下沉）

Args:

t: 时间 (s)
t_e_local: 烟幕起爆时间
consider_validity: 是否检查时间有效性

Returns:

烟幕云团中心在时刻 t 的位置

"""

if consider_validity and (t < t_e_local - 1e-12 or t > t_e_local + T_s + 1e-12):

raise ValueError("t out of cloud active interval")

起爆时刻的位置

C0 = grenade_pos(t_e_local)

dt = t - t_e_local

云团匀速下沉

return C0 + np.array([0.0, 0.0, -v_s * dt])

===== 几何计算函数 =====

def dmin2_point(M: np.ndarray, C: np.ndarray, P: np.ndarray) -> Tuple[float, float]:

"""

计算点 P 到线段 MC 的最小距离平方，以及在线段上的参数位置

Args:

M: 线段起点（导弹位置）
C: 线段终点（云团中心）
P: 目标点

Returns:

(最小距离平方, 在线段上的参数位置[0,1])

"""

a = M - C

d = P - M

d_norm2 = float(np.dot(d, d))

if d_norm2 <= EPS:

return float(np.dot(a, a)), 0.0

计算最近点在线段上的参数位置

s_star = -float(np.dot(a, d)) / d_norm2

if s_star <= 0.0:

最近点是起点 M

return float(np.dot(a, a)), 0.0

elif s_star >= 1.0:

最近点是终点 C

return float(np.dot(P - C, P - C)), 1.0

else:

最近点在线段内部

val = float(np.dot(a, a) - (np.dot(a, d) ** 2) / d_norm2)

if val < 0 and val > -1e-12:

val = 0.0


```

        return val, s_star
def candidate_points(M: np.ndarray, C: np.ndarray) -> List[np.ndarray]:
    """
    生成真目标圆柱体上的候选点，用于计算最大距离

    Args:
        M: 导弹位置
        C: 云团中心位置

    Returns:
        候选点列表
    """
    candidates = []
    a = M - C
    # 圆柱体顶部和底部的中心点
    Cxy = C[:2]
    Txy = T0[:2]
    dxy = np.linalg.norm(Cxy - Txy)
    zC = C[2]
    # 计算从真目标指向云团中心的水平方向
    if dxy <= 1e-12:
        ux = np.array([1.0, 0.0])
    else:
        ux = (Cxy - Txy) / dxy
    # 顶部和底部的边界点
    P_top = np.array([T0[0] + R_T * ux[0], T0[1] + R_T * ux[1], H_T])
    P_bot = np.array([T0[0] + R_T * ux[0], T0[1] + R_T * ux[1], 0.0])
    candidates.append(P_top)
    candidates.append(P_bot)
    # 计算导弹-云团连线与圆柱体的交点
    a_xy = a[:2]
    Mxy = M[:2]
    A_coef = float(np.dot(a_xy, a_xy))
    B_coef = 2.0 * float(np.dot(Mxy - Txy, a_xy))
    C_coef = float(np.dot(Mxy - Txy, Mxy - Txy) - R_T ** 2)
    if abs(A_coef) < 1e-12:
        # 处理特殊情况: a_xy 接近零向量
        if abs(C_coef) <= 1e-9:
            for z in (0.0, H_T):
                candidates.append(np.array([Mxy[0], Mxy[1], z]))
    else:
        # 解二次方程求交点
        disc = B_coef ** 2 - 4.0 * A_coef * C_coef
        if disc >= -1e-12:
            disc = max(disc, 0.0)
            sqrt_disc = sqrt(disc)

```

```

        lam1 = (-B_coef + sqrt_disc) / (2.0 * A_coef)
        lam2 = (-B_coef - sqrt_disc) / (2.0 * A_coef)
        for lam in (lam1, lam2):
            P = M + lam * a
            if P[2] >= -1e-8 and P[2] <= H_T + 1e-8:
                Pz = min(max(P[2], 0.0), H_T)
                candidates.append(np.array([P[0], P[1], Pz]))

# 添加圆柱体中心点
candidates.append(np.array([T0[0], T0[1], 0.0]))
candidates.append(np.array([T0[0], T0[1], H_T]))

# 添加圆柱体表面采样点
thetas = np.linspace(0, 2.0 * pi, 36, endpoint=False)
hs = np.linspace(0.0, H_T, 5)
for th in thetas[::6]:
    for h in hs:
        candidates.append(np.array([T0[0] + R_T * np.cos(th), T0[1] + R_T * np.sin(th), h]))

# 去除重复点
unique = []
for p in candidates:
    if not any(np.linalg.norm(p - q) < 1e-6 for q in unique):
        unique.append(p)
return unique

# ===== 遮蔽判断函数 =====
def is_fully_covered_at(t: float, debug: bool = False) -> Tuple[bool, float, Dict[str, Any]]:
    """
    判断在时刻 t 导弹是否被烟幕完全遮蔽

    Args:
        t: 时间 (s)
        debug: 是否返回调试信息

    Returns:
        (是否完全遮蔽, 最大距离平方, 调试信息字典)
    """
    t_imp = missile_imp_time()
    # 检查时间有效性
    if t < t_e - 1e-12 or t > min(t_e + T_s, t_imp) + 1e-12:
        return (False, float('inf'), {'reason': 'cloud inactive or t outside flight'})

    # 获取导弹和云团位置
    M = missile_pos(t)
    C = cloud_center_at(t)
    a_norm2 = float(np.dot(M - C, M - C))

    # 如果导弹在云团内部
    if a_norm2 <= R_s ** 2 + 1e-12:
        return (True, a_norm2, {'reason': 'missile inside cloud', 'M': M, 'C': C})

    # 计算所有候选点的最小距离

```

```

candidates = candidate_points(M, C)
dvals = []
for P in candidates:
    d2, sstar = dmin2_point(M, C, P)
    dvals.append((P, d2, sstar))
# 找到最大距离
Dmax = max(d[1] for d in dvals)
is_cov = (Dmax <= R_s ** 2 + 1e-9)
info = {'M': M, 'C': C, 'candidates': dvals}
if debug:
    info['a_norm2'] = a_norm2
return (is_cov, Dmax, info)
def DminusRs2(t: float) -> float:
    """
    计算  $D_{max}^2 - R_s^2$ ，用于边界精化
    Args:
        t: 时间 (s)
    Returns:
         $D_{max}^2 - R_s^2$ 
    """
    t_imp = missile_imp_time()
    if t < t_e - 1e-12 or t > min(t_e + T_s, t_imp) + 1e-12:
        return 1.0
    cov, D, _ = is_fully_covered_at(t)
    return D - R_s ** 2
def refine_boundary(lo: float, hi: float, entering_left: bool, max_iter: int = 60) -> float:
    """
    使用二分法精化遮蔽边界时间
    Args:
        lo: 时间下限
        hi: 时间上限
        entering_left: 是否从左边界进入遮蔽区
        max_iter: 最大迭代次数
    Returns:
        精化后的边界时间
    """
    t_imp = missile_imp_time()
    lo = max(0.0, lo)
    hi = min(hi, min(t_e + T_s, t_imp))
    flo = DminusRs2(lo)
    fhi = DminusRs2(hi)
    # 检查边界条件
    if entering_left:
        if flo <= 0 and fhi <= 0:

```

```

        return lo
    if flo > 0 and fhi > 0:
        return hi
    else:
        if flo >= 0 and fhi >= 0:
            return hi
        if flo < 0 and fhi < 0:
            return lo
# 二分法迭代
for _ in range(max_iter):
    mid = 0.5 * (lo + hi)
    fmid = DminusRs2(mid)
    if abs(fmid) <= 1e-7:
        return mid
    if entering_left:
        if fmid > 0:
            lo = mid
        else:
            hi = mid
    else:
        if fmid <= 0:
            lo = mid
        else:
            hi = mid
    if hi - lo < REFINE_TOL:
        return 0.5 * (lo + hi)
return 0.5 * (lo + hi)
def find_full_coverage_intervals(t_start: float, t_end: float, step: float = TIME_STEP) -> List[Tuple[float, float]]:
    """
    查找完全遮蔽的时间区间
    Args:
        t_start: 开始时间
        t_end: 结束时间
        step: 时间步长
    Returns:
        完全遮蔽的时间区间列表
    """
    if t_start >= t_end:
        return []
# 时间扫描
times = np.arange(t_start, t_end + step / 2.0, step)
flags = []
for t in times:

```

```

        cov, D, _ = is_fully_covered_at(t)
        flags.append(bool(cov))
# 识别遮蔽区间
intervals = []
in_interval = False
cur_start = None
for k, t in enumerate(times):
    s = flags[k]
    if s and not in_interval:
        in_interval = True
        cur_start = t
    elif not s and in_interval:
        in_interval = False
        cur_end = times[k - 1]
        intervals.append((cur_start, cur_end))
        cur_start = None
if in_interval:
    intervals.append((cur_start, times[-1]))
# 精化区间边界
refined = []
for (a, b) in intervals:
    left_bracket_lo = max(a - step, t_start)
    left_bracket_hi = a
    right_bracket_lo = b
    right_bracket_hi = min(b + step, t_end)
    left = refine_boundary(left_bracket_lo, left_bracket_hi, entering_left=True)
    right = refine_boundary(right_bracket_lo, right_bracket_hi, entering_left=False)
    if right - left >= 1e-6:
        refined.append((left, right))
return refined

# ===== 主函数 =====
def main():
    """
    主函数：执行烟幕遮蔽分析
    """
    t_imp = missile_imp_time()
    t_scan_start = max(t_e, 0.0)
    t_scan_end = min(t_e + T_s, t_imp)
    print("==== 参数摘要 ===")
    print(f"真目标下底圆心 T0 = {T0}, 半径 R_T = {R_T} m, 高度 H_T = {H_T} m")
    print(f"导弹 M1 初始 M1_0 = {M1_0}, 速度 v_M = {v_M} m/s")
    print(f"无人机 FY1 初始 FY1_0 = {FY1_0}, 速度 v_U = {v_U} m/s (平飞)")
    print(f"投放时刻 t_rel = {t_rel:.3f} s, 引信延迟 Δt = {delta_td:.3f} s, 起爆 t_e = {t_e:.3f} s")
    print(f"烟幕半径 R_s = {R_s} m, 下沉速 v_s = {v_s} m/s, 有效时长 T_s = {T_s} s")

```

```

print()
print(f"导弹到达假目标时间 t_imp = {t_imp:.3f} s")
print(f"云团活跃时间窗口: [{t_e:.3f}, {t_e + T_s:.3f}] s")
print()
if __name__ == "__main__":
    main()

```

问题二代码（核心部分）

```

import numpy as np
import math
import time
from dataclasses import dataclass, field
from typing import Tuple, List, Optional, Dict
from functools import lru_cache
from tqdm import tqdm

def build_target_mesh(P: Params) -> Tuple[np.ndarray, np.ndarray]:
    """根据精度模式构建目标网格"""
    if P.precision == "balanced":
        small = _mesh_cylinder_surface(P, 24, 4, 24, 3)
        full = _mesh_cylinder_surface(P, 72, 12, 72, 8)
    elif P.precision == "high":
        small = _mesh_cylinder_surface(P, 32, 6, 32, 4)
        full = _mesh_cylinder_surface(P, 128, 18, 128, 10)
    else:
        small = _mesh_cylinder_surface(P, 48, 8, 48, 6)
        full = _mesh_cylinder_surface(P, 192, 24, 192, 12)
    return small, full

def phi_value_vec(t: float, pl: Plan, P: Params, tgt_pts: np.ndarray) -> float:
    """计算时刻 t 的遮蔽效果评估值（向量化实现）"""
    M = M_of_t(t, P) # 导弹位置
    C = C_of_t(t, pl, P) # 烟幕中心位置
    # 检查烟幕是否低于地面
    if P.enforce_nonnegative_z and (C[2] < 0.0):
        return 1e18
    a = C - M # 烟幕中心到导弹的向量
    A2 = float(np.dot(a, a)) # 向量长度的平方
    D = tgt_pts - M # 目标点到导弹的向量
    d2 = np.einsum('ij,ij->i', D, D) # 各目标点到导弹距离的平方
    ad = np.einsum('ij,j->i', D, a) # 点积
    eps = 1e-12
    s_star = ad / np.maximum(d2, eps) # 投影参数
    # 三种情况的预计算
    vall = np.full_like(d2, A2) # s_star <= 0 的情况

```

```

val3 = np.einsum('ij,ij->i', tgt_pts - C, tgt_pts - C) # s_star >= 1 的情况
val2 = A2 - (ad * ad) / np.maximum(d2, eps) # 0 < s_star < 1 的情况
# 根据 s_star 值选择对应的计算结果
m1 = (s_star <= 0.0)
m3 = (s_star >= 1.0)
m2 = (~m1) & (~m3)
vals = np.empty_like(d2)
vals[m1] = val1[m1]
vals[m3] = val3[m3]
vals[m2] = val2[m2]
# 返回最大遮蔽效果值
return float(np.max(np.maximum(vals, 0.0)))

def mask_indicator_dual(t: float, pl: Plan, P: Params, tgt_small: np.ndarray, tgt_full: np.ndarray) -> Tuple[bool, float]:
    """双重精度遮蔽检测"""
    rs2 = P.Rs ** 2
    # 先用低精度网格快速检测
    phi_s = phi_value_vec(t, pl, P, tgt_small)
    if phi_s > rs2 + P.eps_switch:
        return False, (phi_s - rs2)
    # 如果需要, 再用高精度网格精确检测
    phi_f = phi_value_vec(t, pl, P, tgt_full)
    return (phi_f <= rs2), (phi_f - rs2)

def refine_root_bisect(tL: float, tR: float, fL: float, fR: float, pl: Plan, P: Params, tgt_small: np.ndarray, tgt_full: np.ndarray) -> Optional[float]:
    """使用二分法精化根的位置"""
    if np.isnan(fL) or np.isnan(fR) or (fL * fR) > 0:
        return None
    a, b = tL, tR
    fa, fb = fL, fR
    # 二分法迭代
    for _ in range(80):
        m = 0.5*(a+b)
        _, fm = mask_indicator_dual(m, pl, P, tgt_small, tgt_full)
        # 收敛条件检查
        if abs(fm) < P.edge_tol or (b-a) < 1e-3:
            return m
        # 更新区间
        if fa * fm <= 0:
            b, fb = m, fm
        else:
            a, fa = m, fm
    return 0.5*(a+b)

```

```

def mask_intervals_event_driven(pl: Plan, P: Params, tgt_small: np.ndarray, tgt_full: np.ndarray) ->
Tuple[List[Tuple[float,float]], float]:
    """事件驱动的遮蔽区间计算"""
    tL = max(0.0, pl.t_e)
    tR = min(P.t_impact(), pl.t_e + P.Ts)

    if tL >= tR:
        return [], 0.0
    # 获取事件节点
    knots = compute_event_knots(pl, P, tgt_small)
    intervals: List[Tuple[float,float]] = []
    inside = False
    t_start = None
    cache: Dict[float, Tuple[bool,float]] = {} # 缓存计算结果
    def get_val(t: float) -> Tuple[bool,float]:
        """带缓存的遮蔽检测"""
        if t in cache:
            return cache[t]
        val = mask_indicator_dual(t, pl, P, tgt_small, tgt_full)
        cache[t] = val
        return val
    # 遍历所有时间区间
    for i in range(len(knots)-1):
        a, b = float(knots[i]), float(knots[i+1])
        if b <= a:
            continue
        in_a, f_a = get_val(a)
        in_b, f_b = get_val(b)
        # 处理符号变化（遮蔽状态改变）
        if (f_a * f_b) < 0:
            t_cross = refine_root_bisect(a, b, f_a, f_b, pl, P, tgt_small, tgt_full)
            if t_cross is not None:
                if f_a <= 0:
                    # 从遮蔽变为非遮蔽
                    intervals.append((t_start if t_start is not None else a, t_cross))
                    inside = False
                    t_start = None
                else:
                    # 从非遮蔽变为遮蔽
                    inside = True
                    t_start = t_cross
            continue
        # 处理区间内状态一致的情况
        if in_a and in_b:

```



```

        if not inside:
            inside = True
            t_start = a
        else:
            if inside:
                intervals.append((t_start if t_start is not None else a, a))
                inside = False
                t_start = None
# 处理最后一个区间
if inside and (t_start is not None):
    intervals.append((t_start, float(knots[-1])))
# 合并重叠区间并计算总遮蔽时间
merged = union_intervals(intervals)
total = float(sum(b - a for a, b in merged))
return merged, total

def delta_upper_bound(P: Params, t_e: float) -> float:
    """计算引信延迟时间的上界"""
    ub = t_e
    if P.enforce_nonnegative_z:
        # 考虑烟幕不能低于地面的约束
        term = 1800.0 - P.vs * P.Ts
        if term <= 0:
            ub = 0.0
        else:
            ub = min(ub, math.sqrt(2.0 * term / P.g))
    return max(0.0, ub)

def golden_section_max(f, a: float, b: float, tol: float, max_iter: int) -> Tuple[float, float]:
    """黄金分割法求最大值"""
    if b <= a:
        fa = f(a)
        return a, fa
    gr = (math.sqrt(5.0) - 1.0) / 2.0 # 黄金比例
    c = b - gr * (b - a)
    d = a + gr * (b - a)
    fc, fd = f(c), f(d)
    # 黄金分割迭代
    for _ in range(max_iter):
        if abs(b - a) < tol:
            break
        if fc < fd:
            a, c, fc = c, d, fd
            d = a + gr * (b - a)
            fd = f(d)
        else:

```

```

        b, d, fd = d, c, fc
        c = b - gr * (b - a)
        fc = f(c)

    return (c, fc) if fc >= fd else (d, fd)

def best_delta_given_te(psi: float, vU: float, t_e: float, P: Params, tgt_small: np.ndarray, tgt_full: np.ndarray,
incumbent: float = 0.0) -> Tuple[float, float, List[Tuple[float, float]]]:
    """给定航向角、速度和起爆时间，优化引信延迟"""
    UB = min(P.Ts, max(0.0, P.t_impact() - t_e))
    if UB <= incumbent + 1e-9:
        return 0.0, 0.0, []
    Dmax = delta_upper_bound(P, t_e)
    if Dmax <= 0.0:
        return 0.0, 0.0, []
    def f_delta(d: float) -> float:
        """目标函数：遮蔽时间关于引信延迟"""
        pl = Plan(psi=psi, vU=vU, t_e=t_e, Delta=max(0.0, min(Dmax, d)))
        tot, _ = objective_total_time(pl, P, tgt_small, tgt_full)
        return tot
    # 使用黄金分割法优化
    d_star, J_star = golden_section_max(f_delta, 0.0, Dmax, P.golden_tol, P.golden_iter)
    segs: List[Tuple[float, float]] = []
    if J_star > 0.0:
        pl_star = Plan(psi=psi, vU=vU, t_e=t_e, Delta=d_star)
        _, segs = objective_total_time(pl_star, P, tgt_small, tgt_full)
    return d_star, J_star, segs

def optimize_all(P: Params, tgt_small: np.ndarray, tgt_full: np.ndarray) -> Dict:
    """全局优化：遍历所有航向角和速度组合"""
    psi_list = np.linspace(0.0, math.pi, P.n_psi, endpoint=False)
    vU_list = np.linspace(P.vU_min, P.vU_max, P.n_vU)
    best_overall = {"best_total": 0.0, "best_plan": None, "best_intervals": []}
    total_iters = len(psi_list) * len(vU_list)
    bp: Plan = best["best_plan"]
    tot = best["best_total"]
    print("总遮蔽时长: %.3f s" % tot)
    print("航向角 psi (弧度): %.6f" % bp.psi)
    print("FY1 速度 vU: %.3f m/s" % bp.vU)
    print("起爆时刻 t_e: %.3f s" % bp.t_e)
    print("引信延迟 Δ: %.3f s" % bp.Delta)
    print("释放时刻 t_rel=t_e-Δ: %.3f s" % (bp.t_rel()))
    C0 = C0_of_plan(bp, P)
    print("起爆点 C0 ≈ (%.3f, %.3f, %.3f)" % (C0[0], C0[1], C0[2]))
    print("遮蔽区间: ", [(("%.3f, %.3f)" % (a, b)) for a, b in best["best_intervals"]])
    if __name__ == "__main__":

```

```
main()
```

问题三、四、五代码(核心部分)

```
import math
import random
import numpy as np
import pandas as pd
from collections import defaultdict
from tqdm import tqdm

# ===== 物理常数和全局参数 =====
# ===== 工具函数 =====

def unit_vector(v):
    """归一化向量"""
    n = np.linalg.norm(v)
    return v / n if n > 0 else v

def missile_dir(i):
    """计算导弹 i 的飞行方向单位向量"""
    M0 = missile_init[i]
    return unit_vector(fake_target - M0)

def missile_pos(i, t):
    """计算导弹 i 在时间 t 的位置"""
    return missile_init[i] + v_missile * missile_dir(i) * t

def mesh_cylinder_surface(dtheta, dh):
    """生成圆柱体表面网格点"""

    thetas = np.arange(0.0, 2.0 * math.pi, dtheta)
    hs = np.arange(0.0, H_T + 1e-9, dh)

    # 侧面点
    side = np.array([[R_T * math.cos(th), 200.0 + R_T * math.sin(th), h]
                     for th in thetas for h in hs])

    # 底面和顶面
    th_cap = np.arange(0.0, 2.0 * math.pi, dtheta)
    rads = np.linspace(0.0, R_T, max(2, int(R_T / max(dh, 1e-6))))
    cap_bottom = np.array([[r * math.cos(th), 200.0 + r * math.sin(th), 0.0]
                           for th in th_cap for r in rads])
    cap_top = np.array([[r * math.cos(th), 200.0 + r * math.sin(th), H_T]
                        for th in th_cap for r in rads])

    return np.vstack([side, cap_bottom, cap_top])

def line_segment_AABB_intersect(P0, P1, box_min, box_max):
    """判断线段与轴对齐边界框是否相交"""

    tmin, tmax = 0.0, 1.0
    d = P1 - P0
```

```

for i in range(3):
    if abs(d[i]) < 1e-12: # 平行于轴
        if P0[i] < box_min[i] or P0[i] > box_max[i]:
            return False
        else:
            inv = 1.0 / d[i]
            t1 = (box_min[i] - P0[i]) * inv
            t2 = (box_max[i] - P0[i]) * inv
            t_enter, t_exit = min(t1, t2), max(t1, t2)
            tmin, tmax = max(tmin, t_enter), min(tmax, t_exit)
            if tmin > tmax:
                return False
    return (tmin < 1.0) and (tmax > 0.0)
def cloud_geometry_at_time(B, e_time, t):
    """计算烟幕在时间 t 的几何形状"""
    zc = B[2] - cloud_sink_v * (t - e_time)
    if zc - cloud_radius > 0.0: # 球形烟幕
        return 'sphere', np.array([B[0], B[1], zc])
    else: # 立方体烟幕
        half = cube_side / 2.0
        return 'cube', (np.array([B[0] - half, B[1] - half, 0.0]),
                        np.array([B[0] + half, B[1] + half, cube_side]))
def max_phi_sphere_over_targets(M, C, targets, Rs):
    """计算球形烟幕对目标的最大遮蔽效果"""
    a = C - M
    A2 = float(np.dot(a, a))
    D = targets - M
    d2 = np.einsum('ij,ij->i', D, D)
    ad = np.einsum('ij,j->i', D, a)
    eps = 1e-12
    s_star = ad / np.maximum(d2, eps)
    val_to_M = np.full_like(d2, A2, dtype=float)
    val_to_Q = np.einsum('ij,ij->i', targets - C, targets - C)
    val_mid = A2 - (ad * ad) / np.maximum(d2, eps)
    # 分段处理
    m1 = (s_star <= 0.0)
    m3 = (s_star >= 1.0)
    m2 = (~m1) & (~m3)
    vals = np.empty_like(d2, dtype=float)
    vals[m1] = val_to_M[m1]
    vals[m3] = val_to_Q[m3]
    vals[m2] = val_mid[m2]
    return float(np.max(vals - Rs * Rs))
def unit2(v2):

```

```

    """二维向量归一化"""
    n = np.linalg.norm(v2)
    return v2 / n if n > 0 else v2
def point_to_line_distance_xy(p, a, b):
    """计算点到直线的距离 (XY 平面) """
    ap = p - a
    ab = b - a
    ab2 = float(np.dot(ab, ab))
    if ab2 <= 0:
        return float(np.linalg.norm(ap))
    cross = abs(ap[0] * ab[1] - ap[1] * ab[0])
    return cross / math.sqrt(ab2)
# 生成目标点网格
target_pts_sel = mesh_cylinder_surface(dtheta_sel, dh_sel) # 选择网格
target_pts_fine = mesh_cylinder_surface(dtheta_fine, dh_fine) # 精细网格
# ===== 快速分配算法 =====
def quick_assignment(uav_init_xy, missiles_xy, V_MAX=140.0):
    """快速分配无人机到目标或导弹走廊"""
    TARGET_XY = target_center[:2]
    ORIGIN_XY = fake_target[:2]
    # 计算到目标的时间
    t_target = {u: np.linalg.norm(pos[:2] - TARGET_XY) / V_MAX
                 for u, pos in uav_init_xy.items()}
    # 计算到各导弹走廊的时间
    t_line = {
        u: {f"M{i}": point_to_line_distance_xy(pos[:2], missiles_xy[i][:2], ORIGIN_XY) / V_MAX
            for i in (1, 2, 3)}
        for u, pos in uav_init_xy.items()
    }
    assigned = {}
    assigned_corridors = set()
    # 优先分配 FY3 到真目标 (如果存在)
    if "FY3" in uav_init_xy:
        if t_target["FY3"] <= min(t_line["FY3"].values()):
            assigned["FY3"] = "真目标"
    # 贪心分配其他无人机
    cands = []
    for u in uav_init_xy:
        if u in assigned:
            continue
        for m in ("M1", "M2", "M3"):
            cands.append((t_line[u][m], u, m))
    for _, u, m in sorted(cands):
        if u in assigned:

```

```

        continue

    if m not in assigned_corridors:
        assigned[u] = m
        assigned_corridors.add(m)
    # 确保至少有一个无人机分配真目标
    some_target_taken = any(v == "真目标" for v in assigned.values())
    for u in uav_init_xy:
        if u in assigned:
            continue
        if not some_target_taken:
            assigned[u] = "真目标"
            some_target_taken = True
        else:
            best_m = min(t_line[u], key=t_line[u].get)
            assigned[u] = best_m
    return assigned, t_target, t_line
# 执行快速分配
assignment_all, t_tar, t_lin = quick_assignment(uav_init, missile_init)
print("=== 快速分配（下界） ===")
for u in sorted(assignment_all.keys()):
    print(f'{u} -> {assignment_all[u]} (to Target ~ {t_tar[u]:.2f}s; '
          f'to M1/M2/M3 ~ {t_lin[u]["M1"]:.2f}/{t_lin[u]["M2"]:.2f}/{t_lin[u]["M3"]:.2f}s)')
print()
# 初始化优先级映射
UAV_PRIORITY_ASSIGNED = defaultdict(dict)
for u, target in assignment_all.items():
    if u not in uav_names:
        continue
    if target == "真目标":
        UAV_PRIORITY_ASSIGNED[u] = {1: 1.0, 2: 1.0, 3: 1.0} # 均匀优先级
    else:
        mi = int(target[1])
        UAV_PRIORITY_ASSIGNED[u][mi] = ASSIGNMENT_WEIGHT # 分配权重
# ===== 候选事件生成 =====
print("阶段 1/7: 生成候选事件（含投放 r 与引爆延迟 Δ）...")
candidates = []
cid = 0
for uav in tqdm(uav_names, desc="候选-遍历 UAV", leave=True):
    p0 = uav_init[uav]
    h_j = p0[2]
    # 基础方向向量（指向目标）
    base_dir = unit_vector(np.array([target_center[0] - p0[0], target_center[1] - p0[1], 0.0]))
    ang_base = math.atan2(base_dir[1], base_dir[0])
    # 生成多个航向角度

```

```

heading_angles = ang_base + np.radians(angle_offsets)
heading_vecs = [np.array([math.cos(a), math.sin(a), 0.0]) for a in heading_angles]
# 最大飞行时间（自由落体）
max_flight = math.sqrt(2.0 * h_j / g)
tfs = np.arange(0.5, max_flight + 1e-9, tfs_step)
# 生成所有参数组合
for v_j in speed_candidates:
    for hv in heading_vecs:
        for r in r_candidates:
            P_r = p0 + v_j * hv * r # 投放点
            for Delta in tfs:
                e = r + Delta # 起爆时间
                if e > Tmax: # 超出最大时间
                    continue
                # 起爆点（考虑重力）
                B = P_r + v_j * hv * Delta + np.array([0.0, 0.0, -0.5 * g * Delta * Delta])
                if B[2] < 0.0: # 低于地面
                    continue
                candidates.append({
                    'id': cid, 'uav': uav, 'heading': hv, 'v_j': v_j,
                    'r': r, 'Delta': Delta, 'e': e, 'P_r': P_r, 'B': B
                })
                cid += 1
print(f'候选事件总数: {len(candidates)}')
# ===== 预筛选候选事件 =====
print("阶段 2/7: 预筛选候选事件...")
kept = []
sparse_idx = np.linspace(0, len(target_pts_sel) - 1, min(36, len(target_pts_sel))).astype(int)
time_grid_sel = np.arange(0.0, Tmax + 1e-9, dt_sel)
def coarse_masked_at(c, i, t):
    """粗略判断候选事件 c 在时间 t 是否对导弹 i 有遮蔽效果"""
    if not (c['e'] <= t <= c['e'] + cloud_effective_time):
        return False
    zc = c['B'][2] - cloud_sink_v * (t - c['e'])
    if zc - cloud_radius <= 0.0: # 烟幕已落地
        return False
    C = np.array([c['B'][0], c['B'][1], zc])
    M = missile_pos(i, t)
    total, ok = 0, 0
    for idx in sparse_idx: # 采样目标点
        Q = target_pts_sel[idx]
        a = C - M
        D = Q - M
        d2 = float(np.dot(D, D))

```

```

        if d2 < 1e-12:
            continue
        ad = float(np.dot(D, a))
        s = ad / d2
        # 计算最小距离平方
        if s <= 0:
            dist2 = float(np.dot(a, a))
        elif s >= 1:
            dist2 = float(np.dot(Q - C, Q - C))
        else:
            dist2 = float(np.dot(a, a) - (ad * ad) / d2)
        total += 1
        if dist2 <= cloud_radius * cloud_radius + 1e-6: # 在遮蔽范围内
            ok += 1
    return (total > 0 and ok / total >= 0.6) # 至少 60%的目标点被遮蔽
# 筛选候选事件
for c in tqdm(candidates, desc="预筛选", unit="cand", leave=True):
    e = c['e']
    if e > Tmax:
        continue
    t1 = int(round(e / dt_sel))
    t2 = int(round(min(e + cloud_effective_time, Tmax) / dt_sel))
    useful = False
    # 检查时间网格点
    for tn in range(t1, t2 + 1, 2):
        t = time_grid_sel[tn]
        for i in (1, 2, 3): # 检查所有导弹
            if coarse_masked_at(c, i, t):
                useful = True
                break
        if useful:
            break
    if useful:
        kept.append(c)
candidates = kept
print(f"筛选后候选数: {len(candidates)}")
candidates_by_id = {c['id']: c for c in candidates}
remaining_ids = set(candidates_by_id.keys())
# ===== 贪心选择算法 =====
print("阶段 3/7: 贪心选择 (锁定(heading,speed) & 分配软偏好 & 自重叠惩罚) ...")
Ntime = len(time_grid_sel)
z_sel = {i: np.zeros(Ntime, dtype=bool) for i in (1, 2, 3)} # 遮蔽状态矩阵
chosen = [] # 已选事件
uav_used_times = defaultdict(list) # 各无人机使用时间

```



```

uav_fixed = {} # 各无人机固定参数
HEADING_TOL = 1e-6
SPEED_TOL = 1e-6
def same_heading(h1, h2, tol=HEADING_TOL):
    """判断两个航向是否相同"""
    return np.linalg.norm(h1 - h2) <= tol
def same_speed(v1, v2, tol=SPEED_TOL):
    """判断两个速度是否相同"""
    return abs(v1 - v2) <= tol
def self_overlap_factor(uav, e_new):
    """计算自重叠惩罚因子"""
    windows = [(c['e'], c['e'] + cloud_effective_time) for c in chosen if c['uav'] == uav]
    for a, b in windows:
        if not (e_new >= b or e_new + cloud_effective_time <= a): # 有时间重叠
            return SELF_OVERLAP_PENALTY
    return 1.0
def incremental_gain_sel(c):
    """计算候选事件 c 的增量增益"""
    e = c['e']
    t1 = int(round(e / dt_sel))
    t2 = int(round(min(e + cloud_effective_time, Tmax) / dt_sel))
    pri_map = UAV_PRIORITY_ASSIGNED.get(c['uav'], {})
    # 地理奖励
    geo_bonus = 1.0
    assigned_obj = assignment_all.get(c['uav'], None)
    if assigned_obj == "真目标":
        if np.linalg.norm(c['B'][:2] - target_center[:2]) <= TARGET_BONUS_THRESH:
            geo_bonus *= ASSIGNMENT_GEO_BONUS
    elif isinstance(assigned_obj, str) and assigned_obj.startswith("M"):
        mi = int(assigned_obj[1])
        d_line = point_to_line_distance_xy(c['B'][:2], missile_init[mi][:2], fake_target[:2])
        if d_line <= CORRIDOR_BONUS_THRESH:
            geo_bonus *= ASSIGNMENT_GEO_BONUS
    # 自重叠惩罚
    overlap_mul = self_overlap_factor(c['uav'], e)
    gain = 0.0
    # 稀缺目标奖励
    scarcity_target = min((1, 2, 3), key=lambda m: np.count_nonzero(z_sel[m]))
    for i in (1, 2, 3): # 对所有导弹
        pri = pri_map.get(i, 1.0) # 优先级权重
        for tn in range(t1, t2 + 1): # 时间网格点
            if z_sel[i][tn]: # 已被遮蔽
                continue
            t = time_grid_sel[tn]

```

```

        if coarse_masked_at(c, i, t): # 有遮蔽效果
            g_inc = pri * geo_bonus * overlap_mul * dt_sel
            if i == scarcity_target: # 稀缺目标额外奖励
                g_inc *= 1.2
            if c['uav'] == "FY5" and i == 3: # FY5 对 M3 额外奖励
                g_inc *= FY5_M3_EXTRA
            gain += g_inc

    return gain
# 贪心选择主循环
pbar_g = tqdm(total=len(remaining_ids), desc="贪心-评估进度", leave=True)
while True:
    best_gain, best_id = 0.0, None
    for cid in list(remaining_ids):
        c = candidates_by_id[cid]
        uav = c['uav']
        r_c = c['r']
        # 检查参数一致性
        if uav in uav_fixed:
            fix = uav_fixed[uav]
            if (not same_heading(c['heading'], fix['heading'])) or (not same_speed(c['v_j'], fix['v'])):
                remaining_ids.discard(cid)
                pbar_g.update(1)
                continue

        # 检查发射次数限制
        if len(uav_used_times[uav]) >= max_per_uav:
            remaining_ids.discard(cid)
            pbar_g.update(1)
            continue

        # 检查发射间隔
        if any(abs(r_c - rr) < min_interval_same_uav - 1e-9 for rr in uav_used_times[uav]):
            remaining_ids.discard(cid)
            pbar_g.update(1)
            continue

        # 计算增益
        gval = incremental_gain_sel(c)
        pbar_g.update(1)
        if gval > best_gain + 1e-12:
            best_gain, best_id = gval, cid
    if best_id is None or best_gain <= 1e-12: # 无更优解
        break
    # 添加最佳候选
    best_c = candidates_by_id[best_id]
    chosen.append(best_c)
    # 锁定无人机参数

```

```

if best_c['uav'] not in uav_fixed:
    uav_fixed[best_c['uav']] = {'heading': best_c['heading'], 'v': best_c['v_j']}
    # 移除不一致的候选
    to_remove_mismatch = set()
    for cid2 in list(remaining_ids):
        c2 = candidates_by_id[cid2]
        if c2['uav'] == best_c['uav']:
            if (not same_heading(c2['heading'], best_c['heading'])) or (not same_speed(c2['v_j'],
best_c['v_j'])):
                to_remove_mismatch.add(cid2)
    remaining_ids -= to_remove_mismatch
    # 更新遮蔽状态
    e = best_c['e']
    t1 = int(round(e / dt_sel))
    t2 = int(round(min(e + cloud_effective_time, Tmax) / dt_sel))
    for i in (1, 2, 3):
        for tn in range(t1, t2 + 1):
            if not z_sel[i][tn] and coarse_masked_at(best_c, i, time_grid_sel[tn]):
                z_sel[i][tn] = True
    # 更新使用记录
    uav_used_times[best_c['uav']].append(best_c['r'])
    # 移除冲突候选
    to_remove = {best_id}
    for cid2 in list(remaining_ids):
        c2 = candidates_by_id[cid2]
        if c2['uav'] == best_c['uav']:
            if abs(c2['r'] - best_c['r']) < min_interval_same_uav - 1e-9: # 间隔冲突
                to_remove.add(cid2)
            if len(uav_used_times[best_c['uav']]) >= max_per_uav: # 次数超限
                to_remove.add(cid2)
    remaining_ids -= to_remove
pbar_g.close()
print(f"贪心已选事件数: {len(chosen)}")
# ===== 局部搜索优化 =====
print("阶段 4/7: 局部搜索优化 (锁定一致) ...")
def evaluate_score_sel(z):
    """计算遮蔽分数"""
    s = 0.0
    for i in (1, 2, 3):
        s += np.count_nonzero(z[i]) * dt_sel # 积分遮蔽时间
    return s
def rebuild_z_from_solution(sol):
    """从解决方案重建遮蔽状态"""
    z = {i: np.zeros(Ntime, dtype=bool) for i in (1, 2, 3)}

```

```

    for c in sol:
        e = c['e']
        t1 = int(round(e / dt_sel))
        t2 = int(round(min(e + cloud_effective_time, Tmax) / dt_sel))
        for i in (1, 2, 3):
            for tn in range(t1, t2 + 1):
                if not z[i][tn] and coarse_masked_at(c, i, time_grid_sel[tn]):
                    z[i][tn] = True

    return z
best_solution = list(chosen)
z_sel = rebuild_z_from_solution(best_solution)
best_score = evaluate_score_sel(z_sel)
# 准备候选池
picked_ids = set(c['id'] for c in best_solution)
uav_pool = defaultdict(list)
for c in candidates:
    if c['id'] not in picked_ids:
        uav_pool[c['uav']].append(c)
# 局部搜索迭代
for _ in tqdm(range(LOCAL_SEARCH_ITERS), desc="局部搜索迭代", leave=True):
    if not best_solution:
        break
    # 随机选择一个事件进行替换
    c_old = random.choice(best_solution)
    uav = c_old['uav']
    pool = uav_pool[uav]
    if not pool:
        continue
    # 过滤符合固定参数的候选
    if uav in uav_fixed:
        fix = uav_fixed[uav]
        pool_filtered = [x for x in pool if same_heading(x['heading'], fix['heading']) and same_speed(x['v_j'],
fix['v_j'])]
    else:
        pool_filtered = pool
    # 采样邻域候选
    neighbors = []
    for __ in range(NEIGHBOR_SAMPLES):
        if not pool_filtered:
            break
        c_try = random.choice(pool_filtered)
        if abs(c_try['r'] - c_old['r']) <= 6.0: # 时间相近的候选
            neighbors.append(c_try)
    # 尝试替换

```

```

improved = False
for c_new in neighbors:
    tmp = [x for x in best_solution if x is not c_old]
    uav_times_tmp = [x['r'] for x in tmp if x['uav'] == uav]
    # 检查约束
    if len(uav_times_tmp) >= max_per_uav:
        continue
    if any(abs(rt - c_new['r']) < min_interval_same_uav - 1e-9 for rt in uav_times_tmp):
        continue
    tmp.append(c_new)
    z_tmp = rebuild_z_from_solution(tmp)
    score_tmp = evaluate_score_sel(z_tmp)
    if score_tmp > best_score + 1e-9: # 找到更优解
        best_solution = tmp
        best_score = score_tmp
        z_sel = z_tmp
        picked_ids.discard(c_old['id'])
        picked_ids.add(c_new['id'])
        # 更新固定参数和候选池
        if uav not in uav_fixed:
            uav_fixed[uav] = {'heading': c_new['heading'], 'v': c_new['v_j']}
            uav_pool[uav] = [x for x in pool if same_heading(x['heading'], c_new['heading']) and
same_speed(x['v_j'], c_new['v_j'])]
        else:
            uav_pool[uav] = [x for x in pool if x['id'] != c_new['id']] + [c_old]
        improved = True
        break
chosen = best_solution
print(f'局部搜索后事件数: {len(chosen)}")
# ===== 容量补齐 =====
print("阶段 5/7: 容量补齐 (尽量每机 3 发, 锁定一致 + 缺口优先 + 自重叠惩罚) ...")
picked_ids = set(c['id'] for c in chosen)
def mark_cover_with(c, z_mask):
    """标记候选事件的遮蔽效果"""
    e = c['e']
    t1 = int(round(e / dt_sel))
    t2 = int(round(min(e + cloud_effective_time, Tmax) / dt_sel))
    for i in (1, 2, 3):
        for tn in range(t1, t2 + 1):
            if not z_mask[i][tn] and coarse_masked_at(c, i, time_grid_sel[tn]):
                z_mask[i][tn] = True
z_sel = rebuild_z_from_solution(chosen)
# 为每架无人机补齐发射次数
for uav in uav_names:

```

```

uav_events = [c for c in chosen if c['uav'] == uav]
# 如果没有事件但有固定参数，跳过初始化
if not uav_events and uav in uav_fixed:
    pass
# 初始化固定参数（如果尚未固定）
if uav not in uav_fixed:
    pool0 = [candidates_by_id[cid] for cid in list(remaining_ids) if candidates_by_id[cid]['uav'] == uav]
    best_seed = None
    best_seed_gain = 0.0
    # 选择最佳初始候选
    for c in pool0[:200]:
        gval = incremental_gain_sel(c)
        if gval > best_seed_gain + 1e-12:
            best_seed_gain, best_seed = gval, c
    if best_seed is not None:
        chosen.append(best_seed)
        uav_fixed[uav] = {'heading': best_seed['heading'], 'v': best_seed['v_j']}
        picked_ids.add(best_seed['id'])
        if uav not in uav_used_times:
            uav_used_times[uav] = []
        uav_used_times[uav].append(best_seed['r'])
        mark_cover_with(best_seed, z_sel)
# 补齐发射次数
for _ in range(FILL_PASS_MAX_ITERS):
    count = sum(1 for c in chosen if c['uav'] == uav)
    if count >= max_per_uav: # 已达最大次数
        break
    if uav not in uav_fixed: # 无固定参数
        break
    fix = uav_fixed[uav]
    pool = []
    # 筛选符合要求的候选
    for cid in list(remaining_ids):
        c = candidates_by_id[cid]
        if c['uav'] != uav:
            continue
        if not same_heading(c['heading'], fix['heading']):
            continue
        if not same_speed(c['v_j'], fix['v']):
            continue
        if any(abs(c['r'] - rr) < min_interval_same_uav - 1e-9 for rr in uav_used_times[uav]):
            continue
        pool.append(c)
    # 选择最佳候选

```

```

best_add = None
best_gain = 0.0
for c in pool:
    gval = incremental_gain_sel(c)
    if gval > best_gain + 1e-12:
        best_gain, best_add = gval, c
if best_add is None or best_gain <= 1e-12: # 无合适候选
    break
# 添加候选
chosen.append(best_add)
picked_ids.add(best_add['id'])
uav_used_times[uav].append(best_add['r'])
mark_cover_with(best_add, z_sel)
# 移除冲突候选
to_remove = set()
for cid in list(remaining_ids):
    c2 = candidates_by_id[cid]
    if c2['uav'] == uav:
        if (not same_heading(c2['heading'], fix['heading'])) or (not same_speed(c2['v_j'], fix['v_j')):
            to_remove.add(cid)
        if any(abs(c2['r'] - rr) < min_interval_same_uav - 1e-9 for rr in uav_used_times[uav]):
            to_remove.add(cid)
        if sum(1 for c in chosen if c['uav'] == uav) >= max_per_uav:
            to_remove.add(cid)
remaining_ids -= to_remove
print(f'补齐后事件数: {len(chosen)} (目标是尽量每机 3 发)')
# ===== 精细计算遮蔽区间 =====
print("阶段 6/7: 精算遮蔽区间 (严格判据) ...")
def precise_intervals_for_event_missile(c, i, targets):
    """精确计算候选事件对导弹的遮蔽时间区间"""
    e = c['e']
    B = c['B']
    tL = max(0.0, e)
    tR = min(e + cloud_effective_time, Tmax)
    if tL >= tR:
        return []
    # 时间网格
    Nt = int(math.ceil((tR - tL) / dt_scan_max)) + 1
    t_grid = np.linspace(tL, tR, Nt)
    intervals = []
    for t in t_grid:
        # 烟幕几何
        shape, geom = cloud_geometry_at_time(B, e, t)
        M = missile_pos(i, t)

```

```

        if shape == 'sphere':
            C = geom
            # 计算最大  $\phi$  值
            phi = max_phi_sphere_over_targets(M, C, targets, cloud_radius)
            if phi <= 0.0: # 有遮蔽
                intervals.append((t, t + dt_scan_max))
        else: # 立方体
            box_min, box_max = geom
            # 检查视线是否与立方体相交
            for Q in targets:
                if line_segment_AABB_intersect(M, Q, box_min, box_max):
                    intervals.append((t, t + dt_scan_max))
                    break

    # 合并重叠区间
    if not intervals:
        return []
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for a, b in intervals[1:]:
        if a <= merged[-1][1]:
            merged[-1] = (merged[-1][0], max(merged[-1][1], b))
        else:
            merged.append((a, b))
    return merged

# 计算每个事件的精确遮蔽区间
event_intervals = defaultdict(lambda: defaultdict(list))
for c in tqdm(chosen, desc="精算-事件", leave=True):
    for i in (1, 2, 3):
        intervals = precise_intervals_for_event_missile(c, i, target_pts_fine)
        event_intervals[c['id']][i] = intervals

# ===== 输出结果 =====
print("阶段 7/7: 输出结果...")
uav_events = defaultdict(list)
for c in chosen:
    uav_events[c['uav']].append(c)

# 按无人机和发射时间排序
for uav in uav_names:
    if uav in uav_events:
        uav_events[uav] = sorted(uav_events[uav], key=lambda x: x['t'])

# 生成输出数据
output_data = []
for uav in uav_names:
    if uav not in uav_events:
        continue

```



```

for c in uav_events[uav]:
    output_data.append({
        '无人机': uav,
        '发射时间 (s)': c['r'],
        '投放点 X (m)': c['P_r'][0],
        '投放点 Y (m)': c['P_r'][1],
        '投放点 Z (m)': c['P_r'][2],
        '起爆时间 (s)': c['e'],
        '起爆点 X (m)': c['B'][0],
        '起爆点 Y (m)': c['B'][1],
        '起爆点 Z (m)': c['B'][2],
        '速度 (m/s)': c['v_j'],
        '航向角 (deg)': math.degrees(math.atan2(c['heading'][1], c['heading'][0])),
    })
# 创建 DataFrame 并输出
df = pd.DataFrame(output_data)
print(df.to_string(index=False))
# 输出统计信息
print(f"\n=== 统计信息 ===")
print(f"总发射次数: {len(chosen)}")
for uav in uav_names:
    count = len(uav_events.get(uav, []))
    print(f"{uav} 发射次数: {count}")
# 计算总遮蔽时间
total_cover_time = {1: 0.0, 2: 0.0, 3: 0.0}
for c in chosen:
    for i in (1, 2, 3):
        for a, b in event_intervals[c['id']][i]:
            total_cover_time[i] += (b - a)
print("\n 导弹总遮蔽时间 (s):")
for i in (1, 2, 3):
    print(f"M{i}: {total_cover_time[i]:.2f}")
print("\n 任务完成! ")

```